

`c++` Rust en toda su gloria

Física Computacional, UNAM, 2020.



Contents

1	Introducción	3
1.1	¿Por qué Rust?	3
1.2	El <code>Hola mundo</code>	4
2	Conceptos básicos	4
2.1	Compilación y herramientas básicas	4
2.1.1	Instalación en <code>*nix</code>	4
2.1.2	Instalación en <code>Windows</code>	5
2.2	Pasos de la compilación	5
2.2.1	<i>Lexer</i>	5
2.2.2	El <i>parse</i> y el AST, <i>Asbtract Syntax Tree</i>	6
2.2.3	Expansión de Macros, validación del árbol, <i>linting</i>	6
2.2.4	HIR	7
2.2.5	MIR	7
2.2.6	LLVM	7
2.2.7	Resumen	7
2.3	Instrucciones	8
2.4	Comentarios	8
2.5	Tipos de variable	9
2.5.1	Enteros	9
2.5.2	Flotantes	10
2.5.3	Booleanos	10
2.5.4	Carácter	10
2.5.5	Anexo: Manejo interno de enteros	10
2.5.6	Anexo2: Manejo de números de precisión flotante	13
2.5.7	Cambio de base	14
3	Conceptos no tan básicos	16
3.1	Jerarquía de expresiones	16
3.2	Bloques de código y memoria	16
3.2.1	El <i>stack</i>	17
3.2.2	El <i>heap</i>	17
3.3	Declaración de funciones	18
3.4	Mutabilidad	19
3.5	Movimientos y Propiedad	19
3.6	Préstamos	21
3.7	Tuplas, Arreglos y <i>rebanadas</i> (slices)	22
3.8	Estructuras	23
3.9	Métodos	24
3.10	Enums	25
3.11	Controladores de flujo	25
3.11.1	<code>if</code> , <code>else</code>	25
3.11.2	<code>for</code>	26
3.11.3	<code>while</code>	26
3.11.4	<code>loop</code>	27
3.11.5	<code>match</code>	27
3.11.6	En común	28
3.12	Vectores	28
3.12.1	Métodos útiles	29
3.13	Cadenas	29
3.13.1	<code>str</code>	30
3.13.2	<code>String</code>	30
3.13.3	Métodos útiles de las cadenas	30
3.14	Los macros <code>print</code> , <code>println</code> y <code>format</code>	31

4	Uso avanzado del lenguaje	32
4.1	cargo	32
4.1.1	crates y Cargo.toml	33
4.1.2	Modularidad	34
4.1.3	Documentación	34
4.1.4	Documentando funciones, estructuras, variables y enums	35
4.2	Parámetros genéricos	35
4.3	Enums <i>recargados</i>	37
4.3.1	El enum <code>Option</code>	39
4.3.2	El enum <code>Result</code>	39
4.4	Características	39
4.4.1	Características comunes	40
4.4.2	Nota importante	40
4.5	Escritura de archivos, entrada y salida estándar	41
4.5.1	Entrada estándar	41
4.5.2	Escritura de archivos	41
4.5.3	Lectura de archivos	42
4.6	Cerraduras	43
5	Extras	44
5.1	Graficando con Rust	44

1 Introducción

Está bien, estoy exagerando demasiado con el título. Pero sí hay varias buenas razones por las que se ha decidido que en este curso utilizaremos Rust, pese a que hay otros lenguajes de programación que podrían, sin problema alguno, cumplir con los requerimientos para este curso.

Sin embargo, para nosotros un lenguaje de programación es una herramienta que no sólo debe ser usada, debe ser entendida. Caso contrario, estaríamos cayendo en esa misma situación de la que la gente de *Ciencias* se burla de los ingenieros.

1.1 ¿Por qué Rust?

Como se menciona en el párrafo anterior, nuestro objetivo en este curso es que adicional a entender los métodos numéricos que ayudan a resolver varios problemas, puedan entender la mecánica detrás de cada algoritmo para saber cuánto tiempo toma en su ejecución, cuántos recursos, etc. Rust es un lenguaje que permite conocer todas estas cosas a la perfección, pero para entenderlo, requerimos entender la siguiente lista de conceptos:

- Nivel del lenguaje de programación: hace referencia a la similitud que un lenguaje de programación tiene con los lenguajes que utilizan los humanos, y los que utilizan las máquinas, siendo estos dos los extremos en una escala donde la cercanía con el lenguaje-máquina se identifica por *bajo*, y la similitud con lenguaje humano se identifica como *alto*. Ejemplos de lenguajes de bajo nivel: **Ensamblador**, **C**. Ejemplos de lenguajes de alto nivel: **Python**, **Scala**.
- Lenguajes Compilados/Interpretados: denominaremos a un lenguaje como compilado cuando el código fuente genera código objeto que es ejecutado por la computadora en dos momentos diferentes. Un lenguaje interpretado será aquél cuyo código fuente va leyéndose y traduciéndose a código máquina al momento de la ejecución.
- Lenguajes estática-y-fuertemente tipados: son lenguajes en los que el tipo de variable está ligado a las variables, y se determina al tiempo de la compilación (es decir, en la ejecución ya no hay información nueva, todo se sabe desde la compilación).
- Colector de basura: mecanismo que utilizan algunos lenguajes para liberar memoria de variables que ya no se encuentran en uso.
- Gestor de paquetes: Cualquier lenguaje de programación sólo está completo si permite la integración de códigos ya existentes para no reinventar la rueda. Algunos lenguajes incluyen un gestor de paquetes y bibliotecas para facilitar esto (**npm** para **javascript** con **nodejs**, **pip** para **Python**, y demás).
- Lenguaje Orientado a Objetos, y lenguaje Funcional: un lenguaje orientado a objetos es aquel que basa todo su funcionamiento en *estados*, es decir, variables que van cambiando a través de una serie de instrucciones para llegar a un valor final. Los estados son el centro de este paradigma. En el paradigma funcional, el centro son funciones y los estados no son explícitamente manejados. Iremos a esto, tal vez, con un poco más de detalle posteriormente.

Nota: Queremos hacer una aclaración sobre todos estos conceptos. Resulta que en todos ellos, hay varias sutilezas que no nos permiten decir con absoluta precisión si un lenguaje es interpretado, si es de absoluto bajo nivel, etc. Estos conceptos tampoco están completamente bien definidos, por ejemplo respecto a los lenguajes intepretados y compilados, existen cosas como Java que hacen ambas, y en general la definición no cubre todos los casos. Hay que tomar estos conceptos a la ligereza, pues sirven más que nada para hacer comparativos.

Dados esos conceptos, podemos muy fácilmente decir por qué **Rust** es la elección de éste curso

1. Rust es estática-y-fuertemente tipado. Esto implica que al momento de la ejecución no pueden ocurrir errores por tipos incompatibles, y adicionalmente conocemos la precisión de nuestros cálculos en todo momento. Simplemente generamos un código con menos errores, por diseño.
2. Rust no tiene colector de basura, porque contiene el concepto de *préstamo de variables* /que veremos a detalle posteriormente), lo que permite que **Rust** sepa qué variables se usan en todo momento, y sabe exactamente cuando puede liberar memoria.
3. Al ser un lenguaje compilado, ahorramos tiempo durante la ejecución, obtenemos un desempeño muchas veces superior al de **C++**, pues el compilador de **Rust** nos obliga a llevar buenas prácticas (y hace optimizaciones por nosotros).

4. Rust es un lenguaje de bajo **y alto** nivel. Esta característica es muy, muy buena, porque permite escribir códigos tan sencillos como en **Python**, pero cuando uno requiere irse al detalle máximo para generar optimizaciones, esto está permitido.
5. El gestor de paquetes de **Rust**, **cargo**, es una *belleza*. El mismo gestor permite generar códigos complejos con una estructura estandarizada que facilita el compartir códigos y desarrollar más rápido, además de contar con códigos que envejecen muy bien.
6. El compilador de **Rust** es muy quisquilloso, y por defecto tiene activadas muchas advertencias y recomendaciones de estilo de código que, de nuevo, son buenas prácticas de la ingeniería del software.
7. **Rust** puede usarse como un lenguaje imperativo, mayormente orientado a objetos, o un lenguaje relativamente funcional.
8. La comunidad crece cada día más, y es muy fácil encontrar ayuda del lenguaje (incluso en foros de discusión en vivo)
9. **Rust** atiende un sector muy desatendido de la programación. Lenguajes como **R**, **Python** y **Julia** atienden al sector que busca hacer análisis de datos. Lenguajes como **Scala**, **Go** y **Erlang** atienden al sector que busca hacer aplicaciones. Lenguajes como **Java** y **C#** atienden al sector de gente que quiere ganar mucho dinero. Y **Haskell** atiende a gente que... bueno... traten de no acercarse a ese tipo de gente. Al final, el sector de la gente que hace sistemas, literal sistemas (el kernel de linux, sistemas embebidos, drivers, y todo eso que hace funcionar al mundo) depende de **C** básicamente, o **C++** si uno tiene suerte. Ahí es donde llega **Rust** a superar toda expectativa.
10. Es rápido. Muy rápido.
11. Generará oportunidades laborales enormes.
12. *Bueno está bien ya, la neta también tengo un favoritismo severo.*

Hay algunas otras razones para adorar **Rust**, pero creo que esas son las principales. Y de todos modos el lenguaje no está a discusión, así que vayamos al grano.

1.2 El Hola mundo

¿Qué clase de introducción estaría completa sin un **hola mundo**? Para que no se queden con las ganas, helo aquí:

```
1 fn main() {  
2     println!("Hola, amigos");  
3 }
```

Nota: Claro está, nuestra elección de **Rust** para el curso tiene su justificación, pero nunca está demás saber varios lenguajes de programación. Tampoco está de más hablar varios idiomas ☺

2 Conceptos básicos

2.1 Compilación y herramientas básicas

El ejecutable principal de **Rust** es el compilador, llamado **rustc**. Este no se encuentra por defecto en ninguna de sus máquinas y debe instalarse del sitio oficial de Rust. Recomendamos que instalen la herramienta **rustup**, que permite ir configurando diversas versiones de **rust** (es un lenguaje que evoluciona relativamente rápido, por lo que a veces es necesario cambiar versiones).

2.1.1 Instalación en *nix

En el caso de los sistemas **Unix** o **Linux**, lo normal es ejecutar desde una terminal

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Listo, esto es todo. Seguramente tendrán que cerrar y reabrir la terminal para ver el resultado de la instalación.

2.1.2 Instalación en Windows

Para esto será todo un poco más complicado que eso. A mí me gusta utilizar la versión GNU de **Rust**, la cuál requiere tener **Mingw** instalado para poder compilar algunas cosas en C. Éste puede bajarse de la página [oficial](#), haciendo click normalmente en donde dice **sourceforge**. Esto descargará un ejecutable. Mi recomendación es que cuando les pida la versión de GNU, pongan las siguientes configuraciones

Opción	Valor
Version	8.1.0
Architecture	x86_64
Threads	posix
Exception	seh
Build revision	0

Si hay una versión mas nueva, realmente pueden instalarla. Cuando les pida una ruta, yo optaría por instalar la herramienta en `C:\mingw-w64`, pues las herramientas de GNU no se llevan bien con los espacios que pone **Windows**. Después, hay que añadir la ruta

`C:\mingw-w64\x86_64-8.1.0-posix-seh-rt_v6-rev0\mingw64\bin`

a las variable de entorno **PATH** de Windows, ya sea del sistema o del usuario.

Nota: La ruta puede cambiar dependiendo su ruta de instalación de **mingw**

Pueden probar que la instalación fue correcta abriendo una terminal e intentando ejecutar **g++**. Debería marcarles un error de compilación que dice algo así como **no files**.

Ahora sí, por último, yo recomendaría que instalen una versión pura de **rust** sin rust up, de la página de instalaciones de **rust**, que diga que es **gnu**.

Para compilar cualquier código fuente de **Rust** contenido en un único archivo con extensión **.rs**, basta con ejecutar

`rustc /ruta/al/codigo/fuente.rs`

Lo que generará un ejecutable con el nombre **fuentes**.

2.2 Pasos de la compilación

A diferencia de lo que existe en las notas de C++, ésta vez iremos un poco más al detalle de cómo funciona el compilador, creo que vale la pena.

2.2.1 *Lexer*

El *Lexer* de un lenguaje es quien se encarga de transformar una cadena de texto en un conjunto de **tokens**, que se pueden reconocer mucho más fácilmente por el *parser*, que es la siguiente instancia de la compilación. En este ejemplo, la instrucción

```
1 let x = 10;
```

Se convertiría en algo parecido a

Token	Tipo
let	Token de declaración
x	Nombre de variable
=	Token de asignación
10	Entero de 32 bits con valor inicial 10
;	Token de fin de instrucción

El *lexer* no hace ninguna simplificación funcional de código ni nada, preserva la información del código fuente casi de manera íntegra.

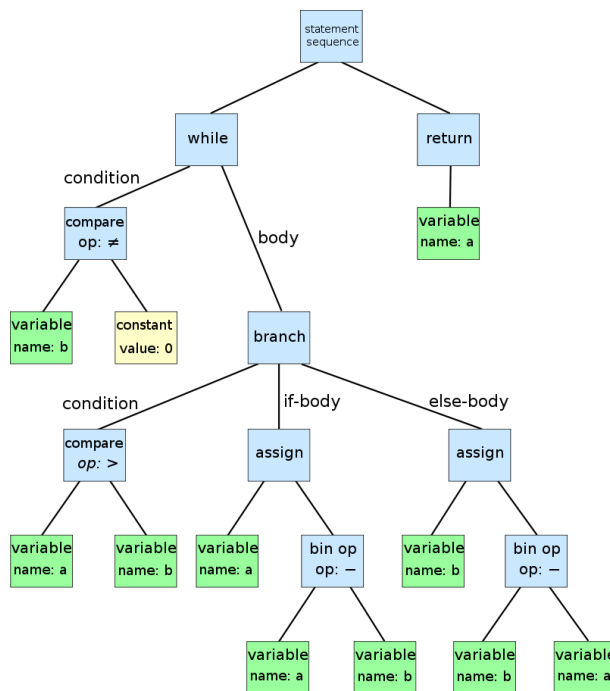
2.2.2 El *parse* y el AST, *Abstract Syntax Tree*

Un árbol de sintáxis abstracta es el primer paso de la compilación, en donde el compilador, `rustc`, se encarga de interpretar el código fuente para colocarlo en un árbol.

Recién robado de Wikipedia, les traigo un ejemplo aproximado de la abstracción de un código en `Rust`. Primero, el código, que es el algoritmo de euclides

```
1 // La entrada del programa ya son a y b
2 while b != 0 {
3     if a > b {
4         a = a - b
5     } else {
6         b = b - a
7     }
8 }
9 return a;
```

Ahora, la abstracción del código



En conjunto, el *lexer* y el *parser* son necesarios para poder obtener las libertades de estructuración del código con las que `Rust` nos bendice, ya lo verán (te estoy viendo a ti `Python`, junto con tu indentación obligatoria).

2.2.3 Expansión de Macros, validación del árbol, *linting*

En esta misma etapa, se comienzan a hacer algunas validaciones de errores en el código, como búsqueda de errores en la estructura del árbol (como ven en el ejemplo de arriba, no puede haber un bloque `if` si no se encuentra acompañado de una condición), y además se corre algo conocido como *linting*, que es una especie de verificación de código que pudiera derivar en un problema, pero no necesariamente lo es.

Aquí mismo ocurre uno de los pasos más increíbles que `Rust` tiene, conocido como **expansión de macros**.

Un **macro** en otros lenguajes de programación, es una palabra que se encuentra en un diccionario de definiciones que mapean a códigos válidos en el lenguaje que se está utilizando. Por ejemplo, `C++` tiene la instrucción `#DEFINE`, que sirve para definir entradas del diccionario. En el caso de `Rust`, el concepto se lleva un paso más adelante, y un macro es un tipo de meta-programación, donde le damos instrucciones al compilador para que *programme* por nosotros. Lo veremos más adelante, pero los macros siempre terminan con un signo de admiración antes de recibir parámetros.

2.2.4 HIR

HIR significa *High-Intermediate-Representation*, básicamente Rust nos quiere decir que tenemos una representación de Rust aún de relativo alto nivel, pero donde ya se han hecho algunas validaciones y re-estructuraciones del código. Al resultado de todas las cosas que han ocurrido en los pasos anteriores, le llamaremos HIR.

2.2.5 MIR

Del HIR deriva el MIR, que como pueden imaginar significa *Middle Intermediate Representation*, en donde se pueden hacer las validaciones estáticas del tipo de variable de cada una de las variables definidas en Rust. Al mismo tiempo, en este paso se realizan algunas simplificaciones para disminuir el número total de instrucciones disponibles al lenguaje. No es el caso de Rust, pero por ejemplo, en otros lenguajes la instrucción `switch` se convierte en un conjunto de `if-else`'s.

Nota: Recuérdeme por favor que cuando estemos viendo ciclos e iteradores, les de un ejemplo de las simplificaciones que se llevan a cabo en este nivel.

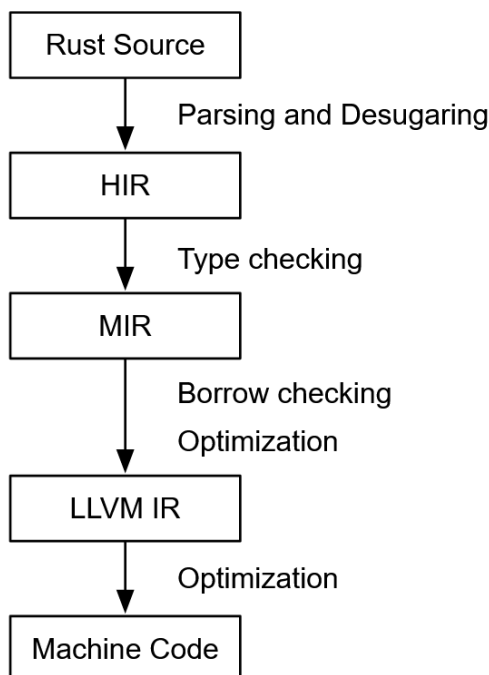
En este paso se revisan también cosas como variables que no está inicializadas y no se utilizan, y lo más importante de todo Rust, aquí se ejecuta el *borrow-checker*, o revisor de préstamos.

2.2.6 LLVM

Último paso. Aquí, estamos listos para convertir el lenguaje en código objeto, en lenguaje máquina. Del MIR brincamos a una representación que puede ser interpretado por LLVM, que es una especie de diccionario que va de representación intermedia de bajo nivel a bloques de código máquina (instrucciones que ya pueden ser directamente interpretadas por el procesador de la computadora). El nombre no es acrónimo alguno, realmente sólo es el nombre que quisieron darle. En este paso también se realizan algunas optimizaciones adicionales de muy bajo nivel, que son comunes a los lenguajes que utilizan LLVM (por ejemplo, Kotlin, Clang C++ ó Swift). Aquí realmente Rust ya no tiene mucho que ver, y obtenemos nuestro ejecutable final.

2.2.7 Resumen

Para que ya no nos enredemos más, veamos el resumen en un simple arbolito



Bueno, para cerrar, si quieren contrastar esto con lo que ocurre, por ejemplo en C++, para referencia les entregaremos el PDF de cursos pasados en donde explicamos lo que hace C++ en los 4 pasos de compilación que tiene (y en donde es un poco más sencillo observar en la computadora cada paso).

2.3 Instrucciones

Rust está compuesto por instrucciones y bloques de código. Las instrucciones terminan con punto y coma (;), a diferencia de Python, y con mucha similitud a C++.

```
1 fn main() {  
2     let x = 10;  
3 }
```

Incluso, tal y como en C++, los espacios y nuevas líneas le son de poca importancia al compilador. Este otro programa compilará sin duda alguna

```
1 fn main() {  
2     let  
3         x  
4         =  
5         10  
6         ;  
7 }
```

... pero traten de evitar hacer ese tipo de barbaries.

Oye pero... No veo el tipo de variable. ¿Acaso nos mentiste? **Confiabamos** en ti...

Pues, también les he dicho que el lenguaje es de alto nivel, y les dije que podrían hacer muchas cosas *casi* como en Python. El tipo de variable ya está asociado a `x`, sólo que ustedes no lo ven. Aunque los ejemplos anteriores creo que se explican por sí solos, aquí explicamos lo que hace una instrucción como la anterior: la palabra `let` es la palabra reservada para hacer declaraciones de variables. La sintaxis de una declaración común en Rust, esta vez incluyendo la especificación del tipo de variable, está compuesta de la siguiente manera:

<u>let</u>	<u>juanito</u>	<u>: i32</u>	<u>= 10</u>	<u>;</u>
Indica una declaración	Nombre de la variable	Tipo de variable	Inicializa la variable	Fin

Con ello, la declaración de nuestra `x` realmente puede verse así

```
1 fn main() {  
2     let x: i32 = 10;  
3 }
```

Incluso les daré un tip. El compilador puede ayudarles a reconocer el tipo de una variable si colocan un tipo que claramente no coincide con lo que hay en el lado derecho de la asignación, por ejemplo

```
1 fn main() {  
2     let x: i32 = "hola";  
3 }
```

les entregará el siguiente error

```
carlos@supercompu:~/> rustc worker.rs  
error[E0308]: mismatched types  
  --> worker.rs:2:18  
2 |         let x: i32 = "hola";  
  |         ~~~~~ expected 'i32', found '&str'  
  |         expected due to this
```

Nota: Oh no, qué diablos es ese *ampersand*, y qué diablos es `str`

2.4 Comentarios

Como buenas prácticas de ingeniería del software que ustedes aprenderán, es importante que sepan cómo se colocan comentarios en los códigos de Rust.

Los comentarios de una única línea se identifican por comenzar con una doble diagonal, `//`. Pero este no es el único tipo de comentario que **Rust** conoce. Cuando se quiere dar una amplia descripción de algo, pueden utilizarse los comentarios de bloque, que comienzan con `/*` y terminan con `*/`. Todo lo que esté contenido entre estos dos identificadores será ignorado por el *lexer*.

```
1 fn main() {
2     // Jaja, mi primer comentario
3     let x = 10;
4     /*
5     El segundo, de bloque
6     */
7 }
```

Hay otros dos tipos de comentario que **rust** reconoce, `///` y `//!`. Su función es la de ayudar al gestor de paquetes, **cargo**, a generar una documentación en **html** estandarizada, limpia y bella para futura referencia o la posibilidad de compartir el código en línea. Estas son características avanzadas del lenguaje que utilizaremos posteriormente.

2.5 Tipos de variable

Rust tiene 4 tipos de variable *elemental*, que son los enteros, los flotantes, los booleanos y los caracteres. Vamos a ir a cada uno de ellos.

2.5.1 Enteros

Los enteros en **Rust** son aquellos números que no tienen parte decimal. La siguiente tabla contiene los diferentes tipos que podemos declarar

Longitud (bits)	Con signo	Sin signo
8	i8	u8
16	i16	u16
32	i32	u32
64	i64	u64
128	i128	u128
arch	isize	usize

Los números con signo se almacenan siguiendo la representación *two's complement* (ver anexo). Una ventaja sobre muchos lenguajes, es que el número de bits está explícitamente escrito en el tipo (en el caso de **c++**, por ejemplo, existen el tipo `int32_t`, pero prácticamente todos utilizan `int`, que no tiene una cantidad de bits estandarizada para todas las plataformas en las que puede compilarse un programa de **C++**).

Otra ventaja sobresaliente de **Rust**, es que permite la asignación de números enteros en diferentes bases, y además permite añadir algo de *azúcar sintáctica* para poder leer números grandes, que consiste en poder meter guiones bajos en el número sin que el compilador se moleste.

Base	Ejemplo
Decimal	10_123
Hexadecimal	0xf3
Octal	0o77
Binaria	0b1101_0101

Adicional, pueden asignar el valor de un carácter **ASCII** a un byte utilizando la notación `b'A'`, que es el ejemplo de asignar el valor 41 a un *byte*, pues 41 en **ASCII** es **A**. Ya con esa información completa, la declaración de un entero puede verse de la siguiente manera

```
1 fn main() {
2     let x: u16 = 0xffff;
3 }
```

Si el número de bytes no coincide, **Rust** nos lo hará saber.

2.5.2 Flotantes

Los números flotantes se identifican exclusivamente por los tipos `f32` y `f64`, donde se puede intuir el número de bits utilizados para su almacenamiento. Los flotantes en `Rust` siguen el estandar IEEE-754 (ver anexo 2).

Al igual que con los enteros, aquí existe un par de maneras en que podemos escribir un flotante: con su representación decimal con punto, o con notación científica, donde una letra `e` denota la expresión *10 a la*.

Notación	Ejemplo
Decimal	10.123
Científica	17e-2

2.5.3 Booleanos

En `Rust`, los booleanos ocupan un *byte* a pesar de representar sólo dos estados, y están identificados por el tipo `bool`.

Por si tienen la duda de esta aparente ineficiencia de memoria, por construcción de las computadoras resulta que la unidad mínima de memoria identificable corresponde a un *byte* y no a un *bit*.

Los valores posibles del booleano se identifican con las palabras `true` y `false` (así en minúsculas, como todos los lenguajes e programación que se respeten... te estoy viendo a ti, `Python`).

2.5.4 Carácter

En `Rust`, los caracteres se identifican por la palabra reservada `char` y ocupan 32 bits, pues están pensados para soportar cualquier carácter de la codificación `utf-8`. La codificación de un carácter no es más que el mapeo de una secuencia específica de bits a una letra (o símbolo) en específico. El hecho de que `Rust` nativa-y-explicitamente soporte la codificación `utf-8` es algo que aprenderán a agradecer muchísimo.

Dicho esto, la asignación de caracteres a una variable del tipo `char` se hace por medio de escribir el carácter que queremos del lado derecho de la asignación rodeado de comillas simples (las comillas dobles están reservadas para las cadenas de caracteres).

```
1 fn main() {  
2     let x = 'a';  
3 }
```

Nota: ¿Cuántos caracteres ocupa la siguiente cadena?

“Soy engañoso”

Seguramente debe ser $12(4bytes) = 48bytes$ ¿no?

No

Las cadenas de caracteres en `Rust` siguen una mecánica diferente. Esa cadena en particular ocuparía 13 *bytes*. Esto lo veremos más adelante ya que hagamos codiguitos con cadenas.

2.5.5 Anexo: Manejo interno de enteros

Podrá parecer trivial, pero el manejo de enteros en una computadora de estados binarios es muy divertido. Creo que para el caso sin signo no hay mucho que decir: si tenemos un número entero almacenado en n -bits, el rango representable por esta secuencia será $[0, 2^n)$, y el mapeo biyectivo no es más que la conversión binaria-decimal. Pero el problema se vuelve un tanto más complicado cuando queremos representar números con signo. Vamos a revisar nuestras opciones, de la más intuitiva, a la menos intuitiva

- Usar el **bit** de la extrema izquierda como un indicador del signo, de modo que si **0b00101** representa el número 5, **0b10101** correspondería a -5. Suena *genial*, pero aparece un problema inminente: la cardinalidad de nuestro conjunto de valores representados por esta secuencia pasa de 2^n a $2^n - 1$ (pues si bien **0b00000** y **0b10000** son estados digitales diferentes, matemáticamente son indistinguibles). No sólo eso, pero hay que darnos cuenta que en el caso sin signo, las operaciones son triviales.

Caso concreto, si queremos realizar la suma de **0b00011** (3) con **0b00111** (7) la operación es *idéntica* a lo que hacen los niños en el *kinder*.

```

...111.
0b00111
+0b00011
-----
0b01010

```

Nota: La línea de puntitos con números es *lo que llevas*, conocido como *carry* en inglés.

Que nos da el tan esperado 10-decimal en su representación binaria. Si intentamos hacer la suma de **0b00001** con **0b10001**, nos encantaría obtener 0, pero...

```

.....1.
0b00001
+0b10001
-----
0b10010

```

...nos da -2. Hay que abandonar el barco.

- Siguiendo opción es hacer un mapeo diferente. Ya es seguro que tenemos que sacrificar un rango de posibles positivos pues la cardinalidad de nuestro conjunto de 5 bits no va a dejar de ser 32. Vamos entonces a definir el negativo de un número que tenga el bit de la extrema izquierda en 0 (es decir, del rango 0 al 15) como la secuencia resultante de ejecutar una **inversión de todos los bits**. Por ejemplo, el negativo de **0b00010** sería **0b11101** (que corresponde al 29). ¿Se solucionó el problema de la suma? A ver, hagamos **0b00010** (2) con **0b11100** (-3):

```

.....
0b00010
+0b11100
-----
0b11110

```

Es un número negativo porque comienza por 1, pero, ¿será el correcto? Si invertimos todos los bits obtenemos **0b00001**, es decir, nuestro resultado **sí es -1**. *Kemosión*. Pero seguimos teniendo dos problemas. La cardinalidad de nuestra representación sigue siendo una unidad inferior a su contraparte sin signo (pues **0b00000** y **0b11111** siguen siendo estados digitales diferentes), y hay también un problema con las operaciones que causan *overflow* y *underflow*. Tomemos como ejemplo la suma de los números 15 (**0b01111**) y -7 (**0b11000**).

```

.111...
0b01111
+0b11000
-----
0b00111

```

Queríamos obtener 8, pero obtuvimos 7... ¿Nos rendimos? ¡**No!** Esta operación es particular porque generó un *overflow*, es decir, se generó un **carry** a la extrema izquierda de nuestro número de 5 bits que ya no tiene dónde existir... pero resulta (por cuestiones del tarot y la astrología) que sí tiene un lugar, y ese lugar es siendo sumado a nuestro resultado, como un 1 en binario

```

...111.
0b00111
+0b00001
-----
0b01000

```

Que es 8, nuestra solución. ¿Pero por qué demonios tuvimos que sumar ese uno al final? La respuesta llegará después de ver nuestro tercer y último método para representar números negativos.

- La solución final es conocida como *2's complement*, y es muy intuitiva. Hay que darnos cuenta que para un número de 5 bits, todas nuestras adiciones son **módulo 32** (siempre son módulo 2^n , con n el número de bits). Es decir, 0b11001 (25) sumado con 0b01111 (15) nos daría como resultado 40, pero ese número no es representable con 5 bits, así que el *carry bit* de la posición 5 se ignora al realizar la operación

```

    ..1111.
    0b11001
+   0b01111
-----
    0b01000

```

Lo que nos da 8, que corresponde a $(25 + 15) \bmod 32$. Podemos aprovecharnos de esta aritmética modular para buscar una manera muy inteligente de representar los negativos.

Imaginemos que queremos realizar la operación 1 menos 1 en binario, y queremos proponer un buen representante del -1. Bueno, podemos notar que, con 5 bits

$$0 \bmod 32 = (1 + (-1)) \bmod 32 = (1 + 31) \bmod 32$$

Ok, entonces para nosotros, 0b11111 (31) es la representación de -1 en binario. Con este mismo cero mágico y haciendo el mismo truco de magia pero con $2 + (-2)$ encontraremos que 0b11110 (30) es la representación de -2. Para no hacer el cuento largo, en lugar de estar haciendo magia para sacar los números negativos, vamos a encontrar una manera más *ingenieril* de calcular los negativos (y a su vez obtener una regla).

Gracias a la aritmética modular tenemos que para a un número entero positivo, $-a \bmod 32 = (32 - a) \bmod 32$. El 32 no lo podemos escribir con 5 bits, pero sí podemos escribir $31+1$ (0b11111+0b00001). Es decir, nuestra regla será que $-a := (31 - a + 1)$.

El cómputo de $b := 31 - a$ puede hacerse de una manera más simple que haciendo el cálculo, gracias a trabajar con una representación binaria. Podrá verse algo tonto, pero si lo escribimos como $a + b = 31 = 2^5 - 1$, conseguir b se vuelve trivial. Usemos a un número de n bits arbitrario y b un binario por descubrir con el mismo número de bits, dado que el resultado de la suma debe ser $2^n - 1$, el resultado debe ser el binario lleno de unos

```

    0b101...001
+   0bxxx...xxx
-----
    0b111...111

```

Si lo vemos *bit* por *bit*, nuestra meta es hacer un XOR entre a y b porque no queremos generar *carry* alguno, y buscamos que el resultado de la suma (*bit a bit*) sea 1 siempre.

Resulta entonces que para un número a de n bits, la suma con su *negado* (cambiar todos los 1 por 0 y los 0 por 1) nos da el número máximo del rango, $2^n - 1$. Con eso ya sabemos que el número b que buscábamos no es más que la negación de a . Así, la regla para obtener un negativo es la siguiente:

1. Toma el número a e invierte todos los *bits*
2. Suma 1 al número que se obtuvo del paso anterior.

Entonces el negativo de 1 sería la negación de 0b00001, que es 0b11110, y al final sumamos 1, con lo que obtenemos el 0b11111 que descubrimos con el *cero mágico*.

El efecto más divertido de esta convención ocurre cuando buscamos el negativo de 0b00000 (0). Resulta que la negación da 0b11111, que al sumar 1 nos da 0b00000 de nuevo. ¡El cero es único!

Sin embargo nos ha ocurrido algo muy extraño, hay un número que se comporta como el cero, el 0b10000 (pues su negativo sería 0b01111+0b00001=0b10000). Este número podría ser el 16, o el -16. Generalmente se toma como el negativo por consistencia pues todos los negativos comienzan con un 1.

Binario	Decimal	Decimal 2's complement
0b00000	0	0
0b00001	1	1
0b00010	2	2
...	...	...
0b01110	14	14
0b01111	15	15
0b10000	16	-16
0b10001	17	-15
0b10010	18	-14
...	...	...
0b11101	29	-3
0b11110	30	-2
0b11111	31	-1

Ésta no es la única manera de manejar enteros, pero es la manera nativa. Para manejo de enteros de precisión arbitraria se suelen almacenar los dígitos en base binaria o decimal, en una cadena de caracteres.

Nota: En el caso de 1's Complement, ocurre que estamos haciendo el mismo mapeo y las mismas adiciones, sólo que tenemos un módulo 31 operando en lugar de un módulo 32 (para el caso de 5 bits). Por eso es que 0b11111 vale 0 (en módulo 31), y por eso es que tenemos que sumar el 1 del carry al final, pues el binario sigue actuando módulo 32, y $(a) \bmod 31 = (a + 1) \bmod 32$, para $31 \leq a < 62$ (que son los casos donde se causa un *overflow*).

2.5.6 Anexo2: Manejo de números de precisión flotante

En el Anexo1 vimos un mecanismo que no nos permite almacenar números con posiciones decimales. ¿Cómo es que la computadora almacena estos números entonces? Dada la naturaleza *finita* de los recursos de la computadora, sólo podemos almacenar números racionales de expresión decimal finita (ahorita veremos cuáles). Esto lo logramos utilizando notación científica, como ejemplo, tomemos los primeros seis dígitos de π , 3.141592, que podemos escribirlos como

$$\underbrace{3141592}_{\text{mantissa}} \times 10^{-6}$$

La parte exponencial es el múltiplo fraccionario pero siempre se escribe en términos de números enteros. A nuestro número en su versión *entera* se le conoce como mantissa.

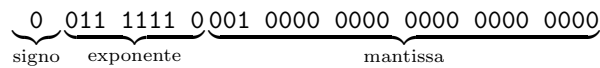
En los números de precisión flotante también hay negativos y positivos, pero esta vez no recurriremos a la “complejidad” de *2's complement*. Esto porque los números de precisión flotante permiten la existencia de otros valores que pueden resultar de operaciones indefinidas.

El reto ahora es guardar la mantissa y el exponente en binario, y para ello utilizaremos como ejemplo la distribución para un flotante de 32 *bits*. Vamos a dividir nuestros 32 *bits* en 3 grupos: el primero es un único *bit* que nos da el signo, el segundo son 8 *bits* que nos dan el exponente escrito como un entero con un desfase de $-2^{m-1} + 1$, y el tercero son los restantes 23, conocidos como *mantissa*.

$$\underbrace{0}_{\text{signo}} \underbrace{000\ 0110\ 0011}_{\text{exponente}} \underbrace{0000\ 0000\ 0000\ 0000\ 0000}_{\text{mantissa}}$$

Hay una situación curiosa aquí. Si tomáramos la mantissa como un número normalito con o sin signo, tendríamos el problema de que, para todos los posibles valores del exponente y el signo, tenemos muchos tipos de 0 (por ejemplo 0×2^{-76}). Además de eso, un número lo podemos escribir en una infinidad de posibilidades con notación científica (π también puede ser 314159200×10^{-8}). Para evitar este problema, se decidió que todos los números deben estar escritos en su notación *normalizada*, que es cuando el primer uno que aparezca en la expresión esté inmediatamente a la izquierda del punto. El número 0.75 en notación normalizada en base dos se escribe 1.1×2^{-1} . Ya que siempre vamos a tener un uno a la izquierda del punto, para obtener más precisión, se decidió asumir que el uno siempre va a estar a la izquierda del punto, y la mantissa representa los dígitos después del punto decimal.

Hagamos el ejercicio con el número 0.5625. El signo es positivo, así que el primer *bit* es cero. Da la casualidad que $0.5625 = 2^{-1} + 2^{-4}$, que se ve en binario como 0.1001, y ya normalizado queda como $1.001 \times 2^{-1} = 1.001 \times 2^{-127+126}$. Escrito como flotante, este número es:



Que da la casualidad en hexadecimal se ve como 0x3f100000. Adicional a esto, hay unos cuantos números reservados. Como ya se habrán dado cuenta, ahora nos es imposible escribir el número 0 (porque en teoría eso daría 2^{-127}), es por esto que se toma como convención que **todos los bits en cero** es igual a 0. La siguiente tabla resume los 4 casos especiales.

Valor	Representación
0	Todos los <i>bits</i> en cero
+inf	Todo el exponente en 1, lo demás en cero
-inf	Signo y todo el exponente en 1, lo demás en cero
NaN	Todo el exponente en 1, <i>mantissa</i> diferente de cero, signo irrelevante.

Para los flotantes de *doble precisión*, el exponente tiene 11 dígitos, la mantissa 52, y el signo 1.

Si tienen curiosidad de verificar estas aseveraciones, en Rust habría que usar códigos no seguros. No se esfuercen de momento en entenderlo, pero el siguiente código les permitirá ver la representación binaria de cualquier flotante

```

1 fn main() {
2     // Reemplacen el lado de la derecha de la asignacion por cualquier
3     // otro valor, por ejemplo 3.141592, f32::INFINITY o f32::NAN
4     let x: f32 = 0.5625;
5     let ptr: *const f32 = &x;
6     unsafe {
7         println!("flags: {:#034b}", *ptr.cast::<u32>());
8     }
9 }

```

2.5.7 Cambio de base

Sobre el cambio de base, tendremos la siguiente expresión

Definición 1 Un número n se expresa en base k , con $2 \leq k$, con dígitos $\{a_i\}$ de la siguiente forma

$$\dots a_2 a_1 a_0 . a_{-1} a_{-2} \dots$$

en donde debe cumplirse que

$$\sum_{-\infty}^{\infty} a_i k^i = n$$

Como un ejemplo un tanto tonto, el número 123.45 puede expresarse como

$$1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

Lo cuál nos dice que en base 10, los dígitos son $a_2 = 1, a_1 = 2, a_0 = 3, a_{-1} = 4, a_{-2} = 5$ y que $\{a_i = 0\}_{i \notin [2, -2]}$. En el caso de base 10 es muy inmediato encontrar los dígitos, ¿Pero en otras bases? No se preocupen, que aquí les viene una explicación sencilla. Para ello, dividiremos el número n que queremos cambiar de base en dos

$$\underbrace{n}_{\text{original}} = \underbrace{\eta}_{\text{parte entera}} + \underbrace{\lambda}_{\text{parte fraccional}}$$

Con la parte entera, podremos calcular los dígitos a la izquierda del punto (es decir $a_0, a_1, \dots$) y con la parte fraccional el resto. Primero vamos con la parte entera. Sin ningún problema podemos suponer que existen números naturales b_0 y r_0 , con las condiciones $0 \leq b_0$ y $0 \leq r_0 < k$ tales que

$$\eta = kb_0 + r_0$$

Notemos que si dividimos la expresión por k , obtenemos lo siguiente

$$\frac{\eta}{k} = b_0 + \frac{r_0}{k}$$

Donde, estrictamente se sigue que b_0 es un natural (más el 0) y podemos ahora ver que r_0 no es más que el residuo de la división del número original por k . Ahora, por las cualidades de b_0 , que es un número natural (más el 0), podemos hacer lo mismo con la parte entera de la división

$$b_0 = kb_1 + r_1$$

Donde b_1 será la parte entera de la división de b_0 por k , y r_1 su residuo. Si conectamos esto en la expresión original, tendremos que

$$\eta = k(kb_1 + r_1) + r_0 = k^2b_1 + kr_1 + r_0$$

Y si lo repetimos una vez más

$$\eta = k^2(kb_2 + r_2) + kr_1 + r_0 = k^3b_2 + k^2r_2 + kr_1 + r_0$$

Ya que podemos repetir esto indefinidamente, nos podemos dar cuenta que $a_0 = r_0, a_1 = r_1$ and so on. Claro está, en cuanto la parte entera de una de las divisiones resulte ser 0, todos los términos a partir de esa división serán 0.

Ahora, vamos con la parte fraccional. Haremos un suposición similar a la de la parte entera, donde sabemos que tienen que existir q_1 y f_1 , con las condiciones $0 \leq q_1$ y $0 \leq f_1 < k^{-1}$, tales que

$$\lambda = k^{-1}q_1 + f_1$$

Multiplicaremos toda esta expresión por k

$$k\lambda = q_1 + kf_1$$

Ahora, dada la condición $0 \leq f_1 < k^{-1}$, sabemos que debe cumplirse que $0 \leq kf_1 < 1$. Esto implica que kf_1 es fraccional. Por el contrario, no tenemos esa garantía con $k\lambda$, la garantía ahí es $0 \leq k\lambda < k$. Podemos darnos cuenta entonces que q_1 tiene que ser la parte entera de $k\lambda$ y kf_1 es la nueva parte fraccional. Dicho eso, entonces podemos decir que

$$kf_1 = k^{-1}q_2 + f_2$$

Que conectándolo en la expresión original, tenemos que

$$\lambda = k^{-1}q_1 + \frac{k^{-1}q_2 + f_2}{k} = k^{-1}q_1 + k^{-2}q_2 + k^{-2}f_2$$

Si repetimos la operación una vez más, llegaremos a que

$$\lambda = k^{-1}q_1 + k^{-1}q_2 + k^{-3}q_3 + k^{-3}f_3$$

Ya que podemos seguir indefinidamente, entonces tenemos que $a_{-1} = q_1, a_{-2} = q_2$ and so on. Si llega a pasar que una de las fracciones multiplicada por k da exclusivamente un entero, el resto de los términos se anularán.

Vamos con un ejemplo: Base 2 ($k = 2$) y el número a convertir es 141.375. Primero vamos con la parte entera

Original	Operación	Parte Entera	Residuo
141	141/2	60	1
60	60/2	30	0
30	30/2	15	0
15	15/2	7	1
7	7/2	3	1
3	3/2	1	1
1	1/2	0	1

De modo que 141 en base 2 se escribe como 1111001. Ahora, haremos lo correspondiente con la parte fraccional del número que queremos escribir en base 2, 0.375.

Original	Operación	Parte Entera	Residuo
0.375	0.375*2	0	0.75
0.75	0.75*2	1	0.5
0.5	0.5*2	1	0

La parte fraccional en binario se escribe entonces como 0.011. Haciendo la suma de la parte entera y fraccional, tenemos que

$$141.375 = 1111001.011$$

3 Conceptos no tan básicos

3.1 Jerarquía de expresiones

Para **Rust** hay muchas ocasiones en que los paréntesis sobran, y por ello es que vale la pena conocer la jerarquía de expresiones que el compilador sigue cuando analiza una instrucción.

Por jerarquía de expresiones nos referimos al orden de evaluación de expresiones. La siguiente tabla contiene esta jerarquía ordenada

Nivel	Operador	Orden
1	Llamadas a métodos	N/A
2	Acceso a campo (.)	Izquierda a derecha
3	Llamadas a funciones	N/A
4	?	N/A
5	- (unitario), *, !, &, &mut	Izquierda a derecha
6	as	Izquierda a derecha
7	*, /, %	Izquierda a derecha
8	+, -	Izquierda a derecha
9	<<, >>	Izquierda a derecha
10	&	Izquierda a derecha
11	^	Izquierda a derecha
12		Izquierda a derecha
13	==, !=, <, >, <=, >=	Requieren paréntesis
14	&&	Izquierda a derecha
15		Izquierda a derecha
16	.., ..=	Requieren paréntesis
17	=, +=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=	Derecha a izquierda
18	return	N/A

Vale la pena conservar esta tabla como referencia, pues aún no conocen todos los operadores (y aunque algunos son intuitivos, otros no).

3.2 Bloques de código y memoria

Un bloque de código en **Rust** es cualquier conjunto de instrucciones contenido entre una llave de apertura y una llave de cierre, por ejemplo en nuestro hola mundo (que no volveremos a poner aquí para tratar de conservar el pdf corto) lo que sigue después de `main()` es una llave de apertura, `{`. Un par de líneas después se encuentra la de cierre, `}`, donde finaliza el bloque de código que contiene una única instrucción.

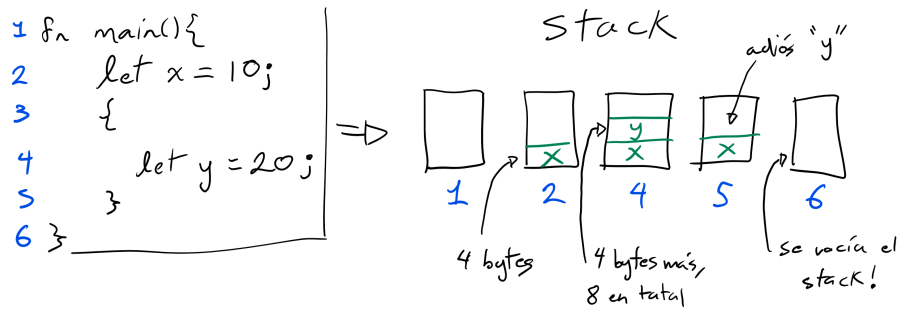
En el caso de nuestro hola mundo, el bloque de código está asociado a una función, pero no siempre tiene que ser así. Por ejemplo, el siguiente código, compilará perfectamente

```
1 fn main() {
2     let x = 10;
3     {
4         let y = 20;
5     }
6 }
```

Es un buen momento para hablar de los dos tipos de memoria que un ejecutable de rust maneja, el *heap* y el *stack*.

3.2.1 El *stack*

El *stack* es un espacio en memoria RAM que tiene la limitante de comportarse justamente como su nombre lo llama (es una estructura de datos que utilizaremos más adelante). Esto quiere decir que es como un tubo en el que podemos meter cosas, pero se cumple que *el último en entrar, es el primero en salir* (se les conoce como estructuras LIFO, *Last in, first out*). Adicional, el tamaño de los elementos del *stack* debe ser conocido al momento de la compilación. No podemos liberar memoria en el orden que se nos da la gana, que podría parecer una limitante terrible, pero esta estructura de datos es muy natural cuando pensamos en la definición de variables en bloques de código. Resulta que el espacio de memoria asociado a las variables declaradas en un bloque de código es un *stack*, y las variables sólo existen en el *stack* mientras el bloque de código no finalice. Tomemos el ejemplo de código más reciente que hicimos, en donde se declaran dos variables en dos bloques de código siendo uno miembro del otro.



En el dibujo anterior se muestra que el *stack* comienza vacío, y luego ingresa la variable *x*. Una vez que entramos al otro bloque de código y se declara *y*, esa *y* existe encima de las variables definidas en el bloque anterior, y dado que si quisieramos borrar *x* tendríamos que eliminar todo lo que está encima de *x*, podemos concluir que se puede hacer referencia a *x* en el bloque contenido, sin miedo a que la variable ya no exista. Ojo que no es el mismo caso respecto a *y* siendo referenciada en el bloque de *main*, pasada la línea 5 (pues el bloque de código finaliza, y estamos autorizados a eliminar toda variable que se haya declarado dentro del bloque pequeño, es decir *y*). Hasta ahora no hay nada en *Rust* que no hagan varios de los otros lenguajes, el *stack* existe del mismo modo en *Java* y *C++*. Sin embargo, tenemos lenguajes como este

```
exists = True
if exists:
    s = "Existo!"
print(s)
```

En el que los bloques de código no tienen una sintaxis única, y ese código puede funcionar o no dependiendo de una variable, con algo que podría evitarse en cualquier lenguaje compilado que respete la sintaxis de los bloques de código (conocidos en inglés como *scopes*).

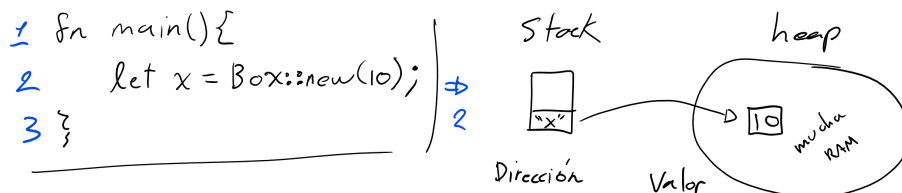
Nota: *Python... ya cámate, po favo*

3.2.2 El *heap*

El otro tipo de memoria es el *heap*, en donde no existe tal regla de orden de creación y eliminación de variables. Hay ciertas *variables* cuyo contenido principal se coloca en el *heap*. Lo veremos a detalle cuando veamos lo que es una estructura. De momento, deben saber que los 4 tipos de variable elemental mencionados con anterioridad van al *stack*, pero podemos pedirle a *Rust* que los coloque en el *heap*. Para ello existe la estructura *Box*. Como aún no es el momento de explicar métodos ni estructuras, sólo deben saber que pueden declarar variables en el *heap* de la siguiente manera

```
1 fn main() {
2     // Creacion de un entero en el heap
3     let x = Box::new(10);
4 }
```

Cabe destacar, que de todos modos, se hace una reserva de espacio en el *stack*, que contiene la dirección de memoria del *heap* en la que se guardó nuestro entero.



Es aquí que los demás lenguajes incurrir en lo que se conoce como *memory leaks*, o fugas de memoria. Las reglas de eliminación de variables del *stack*, en prácticamente todos los lenguajes, no se llevan al *heap* por la simple razón de que múltiples variables pueden apuntar al mismo pedazo de memoria en *heap*, lo cuál hace imposible determinar en qué momento una reserva de memoria puede liberarse. En Rust es imposible que esto ocurra por el concepto de préstamo de variables.

Lo veremos a detalle en la sección de préstamos, pero el siguiente código ejemplifica el uso de una *caja*, y cómo encontrar la dirección de memoria de una variable, y la dirección de memoria del contenido de una caja.

```
1 fn main() {
2     let x = Box::new(10);
3     println!("Dirección de la caja: {:?}", &x);
4     println!("Dirección del contenido: {:?}", &(*x));
5 }
```

3.3 Declaración de funciones

Rust, al usarse de manera imperativa, está compuesto de funciones, y son uno de los bloques fundamentales del lenguaje. Una función no es más que un bloque de código que se antecede por la declaración de un nombre, una serie de argumentos, un valor de retorno y otros parámetros asociados a la visibilidad del bloque. La sintaxis de las funciones es la siguiente

$\underbrace{\text{pub}}_{\text{Visibilidad (opcional)}}$
 $\underbrace{\text{fn}}_{\text{Función}}$
 $\underbrace{\text{nombre_de_la_f}}_{\text{Nombre}}$
 $\underbrace{(\text{arg: i32, otro: f32})}_{\text{argumentos}}$
 $\underbrace{-> \text{u32}}_{\text{Valor de retorno (opcional)}}$
 $\underbrace{\{ }_{\text{Comienzo del bloque}}$

De momento, no nos preocuparemos demasiado por el **pub**, pero es bueno saber que existe. La función puede no obtener argumentos (como en el caso de `main`), o puede contener cualquier cantidad finita de argumentos, que **deben venir acompañados de sus tipos**. Del mismo modo, el valor del retorno es opcional, y es sólo para los bloques que regresan algún valor (por defecto todo bloque de código regresa una *tupla vacía*). Veamos un pequeño ejemplo de una función.

```
1 fn dame_esos_cinco() -> u32 {
2     return 5;
3 }
4 fn main() {
5     println!("{}", dame_esos_cinco());
6 }
```

Las funciones se invocan por su nombre, seguido de un paréntesis y los argumentos que requiere (si es que los lleva).

Ya que los bloques por defecto tienen que regresar *algo*, rust tiene el *azúcar sintáctica* de permitirnos regresar el valor de la última línea de código del bloque si ésta no termina con punto y coma. Es decir, el siguiente ejemplo es equivalente al anterior

```
1 fn dame_esos_cinco() -> u32 {
2     5
3 }
4 fn main() {
5     println!("{}", dame_esos_cinco());
6 }
```

Como una pequeña nota que tal vez favorece el entendimiento del funcionamiento de *Rust*, las funciones que reciben argumentos, prácticamente lo que hacen es declarar variables al principio del bloque de código que compone a la función. Es decir, el siguiente ejemplo de código

```

1 fn sumador(a: i32, b: i32) -> i32 {
2     a + b
3 }
4 fn main() {
5     let x = 10;
6     let y = 20;
7     println!("{}", sumador(x, y));
8 }

```

es el equivalente al siguiente código

```

1 fn main() {
2     let x = 10;
3     let y = 20;
4     println!("{}", {
5         let a = x;
6         let b = y;
7         a + b
8     });
9 }

```

Mantengan esto en mente porque vendrá a ser de utilidad posteriormente.

3.4 Mutabilidad

Por defecto, todas las variables en Rust son *inmutables*, que es algo así como decir que son constantes. El siguiente código de ejemplo, fallará

```

1 fn main() {
2     let x = 10;
3     x = 20;
4 }

```

Ignorando de momento los *warnings* que nos de el compilador, el error deberá verse algo así

```

carlos@supercompu:~/> rustc worker.rs
error[E0384]: cannot assign twice to immutable variable 'x'
--> worker.rs:3:5
2 |     let x = 10;
  |     ~~~~~
  |     |
  |     first assignment to 'x'
  |     help: make this binding mutable: 'mut x'
3 |     x = 20;
  |     ^^^^^ cannot assign twice to immutable variable

```

La sugerencia nos la da el mismo compilador, indicándonos que la palabra *mut* frente a la instrucción de declaración de una variable le avisa a Rust que esa variable está *hecha* para ser cambiada.

```

1 fn main() {
2     let mut x = 10;
3     // Ahora ya no hay error, no hay error, no hay error
4     x = 20;
5 }

```

3.5 Movimientos y Propiedad

Antes de pasar al tema de los préstamos, el núcleo de Rust, hablaremos sobre el concepto de *ser dueños de la memoria*.

En Rust, las variables que hemos visto hasta ahora son dueñas de la información a la que apuntan. Estas variables son responsables de liberar la memoria cuando ya no se utiliza (como lo que ocurre en el *stack* cuando un bloque de código finaliza, por ejemplo). El hecho de que la información tenga un dueño, asegura que la información se borra una única vez

también.

Volvamos un poco al ejemplo del uso de una *caja*, e intentemos compilar el siguiente código

```
1 fn main() {
2     let x = Box::new(10);
3     let y = x;
4     println!("Numerito: {}", x);
5 }
```

Esto, en un lenguaje como *py*... está bien ya, como cualquier lenguaje en general, no causaría ningún tipo de problema. Pero aquí, el *borrow-checker* de *Rust* nos informará que estamos intentando acceder a una variable que ya no tiene información.

```
carlos@supercompu:~/> rustc worker.rs
error[E0382]: borrow of moved value: 'x'
--> worker.rs:3:5
|
2 |     let x = Box::new(10);
|     - move occurs because 'x' has type 'std::boxed::Box<i32>', which does not implement the 'Copy' trait
3 |     let y = x;
|     - value moved here
4 |     println!("Numerito: {}", x);
|                                   ^ value borrowed here after move
```

Lo que ocurrió aquí es que la declaración de *y* implícitamente toma propiedad de lo que *x* contenía. Esto ocurre para poder garantizar que la información contenida en el *heap* se elimine una y verdaderamente una única vez. No más, no menos.

A lo ocurrido entre *y* y *x* se le conoce como un movimiento, y esto **no ocurre con cualquier variable**. Las variables elementales son clonadas por defecto, por su bajo costo, pero prácticamente cualquier variable que tiene algo que ver con el *heap*, necesita a un propietario.

Nótese que esto ocurre en las funciones también

```
1 fn adios(x: Box<i32>) {
2     // ni si quiera tengo que poner codigo
3 }
4
5 fn main() {
6     let x = Box::new(10);
7     adios(x);
8     println!("Numerito: {}", x);
9 }
```

Esto fallará porque la declaración implícita del argumento de la función toma posesión del contenido de *x*. Pero entonces tenemos un pequeño problema... Eso quiere decir que siempre que tengamos una función que requiere acceso a una variable, ¿tendremos que regresar la variable al final de la función?

```
1 fn no_tan_adios(x: Box<i32>) -> Box<i32> {
2     // Hago lo que sea que necesito con x
3     x
4 }
5
6 fn main() {
7     // x tiene que ser mutable porque recibira un "nuevo valor"
8     let mut x = Box::new(10);
9     x = no_tan_adios(x);
10    println!("Numerito: {}", x);
11 }
```

Esto es una solución, claro está, pero nos obligó a hacer la variable mutable de manera innecesaria (pues estamos suponiendo que la función *no_tan_adios* sólo utiliza a *x* para hacer algunos cálculos sin alterar la variable).

3.6 Préstamos

Entre esta sección y la anterior, estamos adentrándonos en las mejores partes de **Rust**. Para quienes hayan usado un lenguaje como **C++**, deben haber peleado al menos una vez con apuntadores. Si tienen curiosidad de entenderlos, pueden consultar el [pdf](#) que hace referencia a este lenguaje.

En **Rust** el concepto de préstamos es muy simple: para cada tipo de variable es posible construir un tipo que en lugar de almacenar un valor, almacena una referencia a un valor. Por ejemplo

```
1 fn main() {
2     let x = 10;
3     {
4         let y: &i32 = &x;
5         println!("Jaja, x me pertenece, {}", *y);
6     }
7     println!("Mentira: v{}", x);
8 }
```

En este caso, aún después de no existir `y`, podemos seguir utilizando `x` pues `y` sólo contenía un préstamo.

Nota: Tres cosas.

1. En el macro `println`, utilizamos un asterisco antes de la `y` para *desreferenciar* nuestra referencia, eso quiere decir, obtener el valor al que hace referencia la variable `y` (si no pusieramos un asterisco, normalmente estaríamos haciendo referencia a una dirección de memoria y no a un valor).
2. El tipo `&i32` no era necesario, pero lo puse por claridad. Como vayan mejorando en el uso del lenguaje, pueden ir jugando con las características de alto nivel del mismo para tener códigos más sucintos.
3. Las referencias pueden tomarse para cualquier tipo de variable, así que ya no necesitamos usar `Box` para pasar un entero a una función.

Pues todo parece indicar que ya hemos solucionado todo problema y lo entendemos todo. Esto debería funcionar perfectamente

```
1 fn debe_funcionar(x: &i32) {
2     // Tomamos equis y le sumamos algo
3     *x += 1;
4 }
5
6 fn main() {
7     // Declaramos x como mut para poder cambiarla :D
8     let mut x = 10;
9     debe_funcionar(&x);
10    println!("Siiiiii{}", x);
11 }
```

¿Qué?

```
carlos@supercompu:~/> rustc worker.rs
error[E0594]: cannot assign to '*x' which is behind a '&' reference
--> worker.rs:3:5
1 | fn debe_funcionar(x: &i32) {
  |                      _____ help: consider changing this to be a mutable reference: '&mut i32'
2 |     // Tomamos equis y le sumamos algo
3 |     *x += 1;
  |     ^^^^^^^ 'x' is a '&' reference, so the data it refers to cannot be written
```

Así es, las referencias en **Rust** también transportan información respecto a la mutabilidad del préstamo. Es decir, yo puedo tener una variable mutable, pero puedo hacer préstamos que no permitan modificar mi variable. Para que la función de arriba funcione, la declaración de la función debe ser la siguiente

```
1 fn debe_funcionar(x: &mut i32) {  
2     *x += 1;  
3 }
```

Y la llamada a la función debe modificarse para pasar una referencia mutable

```
1 debe_funcionar(&mut x);
```

Nota: Intenten explicarme lo que significa entonces la declaración `let mut x: &mut i32 = ...`.

Además de esto, los préstamos no están exentos de reglas. Resulta que dada una variable mutable, podemos generar cualquier cantidad finita de préstamos inmutables y debemos prometer no hacer préstamos mutables mientras existan préstamos inmutables a la variable.

Nota: En realidad nada tienen que prometer. El compilador de **Rust** los obligará de todas formas.

La segunda regla es que sólo puede existir un préstamo mutable a la vez, y sólo uno.

Nota: El compilador de **Rust** es lo suficientemente inteligente para detectar si estos casos se dan, porque el hecho de declarar dos préstamos mutables no quiere decir que se utilicen de manera mutable. Hagan el experimento.

3.7 Tuplas, Arreglos y *rebanadas* (slices)

Una tupla es un conjunto de valores de tipos diferentes. Una tupla de tipos (T1, T2, T3) define en sí un nuevo tipo en **Rust**, por lo que la cantidad de elementos que almacena ya no puede modificarse una vez creada la tupla. Las tuplas se reconocen fácilmente por que la colección de elementos está rodeada de paréntesis.

```
1 fn main() {  
2     // Tupla del tipo '(char, bool)'.  
3     let tupla = ('c', true);  
4 }
```

Uno de los usos más bonitos que tienen las tuplas, es el de servir como herramienta para hacer declaraciones de múltiples variables en una única línea

```
1 fn main() {  
2     let (x, y): (i32, i32) = (10, 20);  
3 }
```

Nota: Ya saben que el tipo lo pongo por claridad.

Así como existen las tuplas, existen también los arreglos, cuya declaración se caracteriza por unos corchetes en lugar de unos paréntesis.

```
1 fn main() {  
2     let arreglo = [10, 20];  
3 }
```

Los arreglos son parecidos a las tuplas en cuanto a que tienen una longitud definida conocida al tiempo de la compilación, pero todos los elementos miembro deben ser del mismo tipo. Esta última restricción permite que la reserva de memoria sea contigua, es decir, todos los elementos están *pegaditos pegaditos*.

Nota: El truco de declarar múltiples variables también se puede hacer con el arreglo.

Si usan el truco del compilador para saber qué tipo de variable contiene `arreglo`, les dirá que el tipo es `[integer; 2]`, que es como se identifican los arreglos de un tipo y una longitud.

Las rebanadas son iguales, pero la longitud no tiene que ser conocida al momento de la compilación.

```
1 fn haz_algo(rebanada: &[i32]) {
2     // Hago algo con el slice aqui
3 }
4
5 fn main() {
6     let arreglo: [i32; 5] = [1, 2, 3, 4, 5];
7     // Un préstamo de un arreglo es una rebanada
8     haz_algo(&arreglo);
9 }
```

Nótese que los *slices* siguen teniendo que cumplir el ser una porción contigua de memoria, por lo que elementos arbitrarios de un arreglo no podrían formar una rebanada, pero cualquier sub-secuencia sin saltos sí lo será. Para extraer una secuencia de un arreglo, existe el operador `..` que a la izquierda toma el índice inicial inclusivo, y a la derecha el índice final no inclusivo, y genera algo conocido como un *rango*.

```
1 fn main() {
2     let arreglo: [i32; 5] = [1, 2, 3, 4, 5];
3     // slice con los elementos [2, 3, 4]
4     haz_algo(&arreglo[1..4]);
5 }
```

También existe el operador `..=`, que sirve para pedir inclusividad en el extremo derecho del rango.

Nota: Los arreglos y todo en Rust está indexado en 0. *Matlab*, *R* y lamentablemente *Julia*, los estoy viendo a ustedes esta vez. Te salvaste, *python*.

Si creen que comenzar en 1 es mejor, recomendaría yo que lean [esto](#).

3.8 Estructuras

Rust no es un lenguaje puramente orientado a objetos, es más bien un lenguaje estructurado. Como tal, entendemos por estructurado que su concepto de *objeto* sólo contempla un conjunto de datos de tipos diferentes que están asociados de alguna manera. Pensemos por ejemplo en una estructura como un punto en $\mathbb{R}^3$, que llamaremos de momento **Punto**. Su definición en Rust, asumiendo que sus coordenadas se identifican por *x*, *y* y *z* sería

```
1 struct Punto {
2     x: f64,
3     y: f64,
4     z: f64
5 }
```

Para volver a darle un poquito de formalidad, la declaración de una estructura tiene la siguiente... estructura

$$\underbrace{\text{pub}}_{\text{Visibilidad (opcional)}} \underbrace{\text{struct}}_{\text{estructura}} \underbrace{\text{NombreDeLaEstructura}}_{\text{Nombre}} \left\{ \underbrace{\text{pub}}_{\text{visibilidad}} \underbrace{\text{foo}}_{\text{nombre}} \underbrace{: \text{i32}}_{\text{tipo}}, \underbrace{\dots}_{\text{más parámetros}} \right\}$$

La estructura puede o no contener parámetros, y cabe resaltar que las declaraciones de estructuras no terminan con punto y coma.

Nota: A diferencia de la declaración de la función, en esta ocasión las palabras del nombre las identifiqué por su comienzo con mayúscula en lugar de separar con guiones bajos. Esto es algo que **Rust** observa. Se le conoce como *Pascal case*, y a la de las funciones *snake case*.

Ahora bien. ¿Cómo declaramos una instancia de una de estas estructuras entonces? Ah pues muy fácil, el código final para declarar un punto sería el siguiente:

```
1 struct Punto {
2     x: f64,
3     y: f64,
4     z: f64
5 }
6
7 fn main() {
8     let p = Punto{x: 0.0, y: 1.0, z: -1.0};
9     println!("Coordenadas_({},{},{})", p.x, p.y, p.z);
10 }
```

Que vaya, me he adelantado un poco aquí pero la manera de acceder a los miembros de una estructura es por medio del operador *puntito* (`.` pa los cuates).

3.9 Métodos

Y bien, ¿Cuál es la diferencia entre una función y un **método**? Los métodos son funciones que se encuentran asociadas a una estructura, y estos pueden requerir una instancia de la estructura o no. Para declarar métodos, se utilizan los bloques de código identificados con la instrucción `impl` de la siguiente manera

```
1 impl Punto {
2     fn saluda() {
3         println!("Hola!");
4     }
5 }
```

Las funciones que se declaran dentro del bloque `impl` siguen la misma sintaxis que una función cualquiera, pero existe la siguiente característica extra: Podemos definir métodos que requieran una instancia de la clase en la que están implementados. Para ello, **proporcionamos como primer argumento de la función una variable con el nombre `self`**, que no requiere tener el tipo especificado pues **Rust** sabe que se trata de una instancia de la estructura que implementa.

```
1 impl Punto {
2     fn grita_coordenadas(&self) {
3         println!("ESTOY_EN_({},{},{})", self.x, self.y, self.z);
4     }
5 }
```

Nota: Por favor, noten que las reglas de préstamos y posesión aplican exactamente de la misma manera para el parámetro `self`, así que si no lo ponen como una referencia, la función *consumirá* la instancia de la clase (es decir, la instancia de la clase se mueve al cuerpo del método una vez que es llamada). Esto a veces es deseable, pero es bueno tener presente que esto ocurre.

Como última observación, los métodos que requieren una instancia de la clase se invocan con el mismo operador *puntito*, seguido del nombre del método y con un paréntesis para proporcionar los argumentos que pudiese requerir el método.

Nota: El primer parámetro, `self`, no se proporciona pues **Rust** lo toma en automático. Del mismo modo, no es necesario colocar ni un *ampersand* ni nada aunque el método tomase una referencia mutable o lo que sea de la instancia del objeto, esto es *azúcar sintáctica* del lenguaje.

Los métodos que están asociados a una estructura pero no requieren una instancia, son conocidos como métodos estáticos y se invocan con el operador `::`, como en **C++**. Ya que no requieren una instancia, ese operador actúa directo sobre el

nombre de la estructura asociada a los métodos. Nuestro supercódigo final quedaría de la siguiente forma:

```
1 struct Punto {
2     x: f64,
3     y: f64,
4     z: f64
5 }
6 impl Punto {
7     fn saluda() { // Metodo estatico
8         println!("Hola!");
9     }
10    fn grita_coordenadas(&self) { // Metodo normalito
11        println!("ESTOY EN {}, {}, {}", self.x, self.y, self.z);
12    }
13 }
14 fn main() {
15     Punto::saluda(); // estatico
16     let p = Punto{x: 0.0, y: 1.0, z: -1.0};
17     p.grita_coordenadas(); // normalito
18 }
```

Nota: Hay un método estático que es muy común ver, conocido como el *constructor* en otros lenguajes. En Rust no existe una regla dura sobre este método, pero suele utilizarse el nombre **new** para denotar un constructor.

3.10 Enums

Un **enum** en cualquier lenguaje no es más que una colección de posibilidades únicas, usualmente identificados por un entero o por una cadena de caracteres. Pero en **rust**, *oh boy...* Son un elemento exageradamente poderoso. Los **enum** en **rust** son una colección de *variantes* únicas entre sí, donde cada variante puede ser desde un simple identificador, hasta una tupla o una estructura anónima. Además de esto, los **enum** también pueden ser proporcionados a la instrucción **impl** para dotarlos de métodos asociados.

```
1 enum Pronostico {
2     Soleado,
3     Nublado,
4     Lluvioso,
5     ConViento
6 }
```

La declaración de un **enum** es parecida a la de una estructura, sólo que en lugar de variables miembro recibe una lista de variantes que deben llevar todas un nombre único. Para acceder a un miembro del **enum**, se utiliza el mismo operador que el de acceso a métodos asociados de una estructura (por ejemplo, **Pronostico::Nublado**).

El uso a fondo de los **enum** los veremos en la sección de uso avanzado, de momento era necesario introducirlos para entender los controladores de flujo en su totalidad.

3.11 Controladores de flujo

Como en cualquier lenguaje, los controladores de flujo definen el comportamiento del programa en función de las variables de las que dependa, y son el bloque fundamental de cualquier lenguaje de programación. En el caso de **Rust**, nos enfocaremos en 4 controladores de flujo (y hablaremos de un 5to sólo para su conocimiento)

3.11.1 if, else

El más esencial, creo yo. Sirve para evaluar una expresión cuyo valor de retorno sea un booleano y ejecutar partes del código de manera condicional. La expresión de la que se alimenta el **if** no tiene por qué estar rodeada de paréntesis.

```

1 fn main() {
2     let x = 10;
3     if x < 100 {
4         println!("No tenemos 100");
5     } else {
6         println!("Tenemos 100 o mas!");
7     }
8 }

```

El bloque `else` que le sigue al `if` se ejecuta si la expresión del `if` no se cumplió (y no es obligatorio que se ponga). Igual que en otros lenguajes, pueden colocarse varias condiciones seriadas con la instrucción `else if`

```

1 fn main() {
2     let x = 10;
3     if x < 100 {
4         println!("No tenemos 100");
5     } else if x < 200 {
6         println!("Tenemos entre 100 y 200!");
7     };
8 }

```

3.11.2 for

Rust es un lenguaje que basa todo su comportamiento en iteradores, a diferencia por ejemplo de C++, en donde la base de los ciclos son los índices.

Nota: Yo sé que C++ tiene ciclos basados en iteradores también, pero no son los que se utilizan con mayor frecuencia.

Esto nos obliga a obtener siempre iteradores sobre los objetos sobre los que queremos iterar, valga la redundancia. No están para saberlo, ni yo para contarlo, pero para eso existe una cosa que se le llama la característica `IntoIter`, que permite extraer un iterador de un objeto, por ejemplo de un arreglo, al llamar el método asociado `into_iter`. Veremos esto a detalle cuando veamos métodos y características.

```

1 fn main() {
2     let array = [1, 2, 3, 4, 5];
3     // Para evitar que el arreglo sea consumido, lo prestamos
4     for element in &array {
5         println!("{}", element);
6     }
7 }

```

Los rangos, generados con el operador `..`, también pueden ser convertidos a un iterador

```

1 fn main() {
2     // El operador de rango ya regresa una referencia
3     for element in 0..10 {
4         println!("{}", element);
5     }
6 }

```

3.11.3 while

El controlador de flujo `while` funciona con una única expresión que debe regresar un booleano, y ejecuta un bloque de código hasta que la expresión regrese `false`

```

1 fn main() {
2     let mut x = 5;
3     while x > 0 {
4         println!("x_no_es_cero");
5         x -= 1;
6     }
7     println!("x_es_cero!");
8 }

```

3.11.4 loop

Más que nada se los pongo como breviarío cultural. La instrucción `loop` equivale a un `while true`. En general, todos los ciclos pueden reducirse a este, acompañado de la instrucción `break`, que permite romper el ciclo.

```

1 fn main() {
2     let mut i = 10;
3     loop {
4         println!("{}", i);
5         i -= 1;
6         if i == 0 {
7             break;
8         }
9     }
10 }

```

Nota: La palabra `break` puede interrumpir cualquier otro tipo de ciclo también.

3.11.5 match

Mi controlador de flujo favorito. La instrucción `match` sirve para hacer un *emparejamiento* de patrones. Por ejemplo, en el caso de un `enum` podemos hacer lo siguiente

```

1 enum Pronostico {
2     Soleado,
3     Nublado,
4     Lluvioso,
5     ConViento
6 }
7 fn main() {
8     let p = Pronostico::Nublado;
9     match p {
10         Pronostico::Soleado => println!("Soleado:"),
11         Pronostico::Nublado => println!("Soleado:v!!!"),
12         Pronostico::Lluvioso => println!("Lluvioso:v!!!!!!"),
13         Pronostico::ConViento => println!("Con_viento!")
14     }
15 }

```

Nota: Si compilan ese código de ejemplo, se darán cuenta que el compilador de `Rust` les advertirá que hay 3 variantes del `enum` que nunca se construyen, sugiriéndoles que las quiten. Gracias, señor `Rust`.

La instrucción `match` también puede usarse para variables con un número mucho mayor de posibilidades, gracias a que permite dar un *alias* a las coincidencias no explícitas

```

1 fn main() {
2     let num = 1;
3     match num {
4         0 => println!("Es_cero,_gracias_a_los_mayas"),
5         1 => println!("Es_uno!"),
6         2 => println!("Es_dos!"),
7         3 => println!("Es_tres!"),
8         mut otro => {
9             otro = otro + 10;
10            println!("Es_otro_numero,_ya_le_puse_10_mas_({})", otro)
11        }
12     }
13 }

```

En el último caso, el valor de `num` se pasó a `otro`, para ser utilizado dentro de un bloque de código (así es, yo abusé un poquito, pero después del operador `=>` puede ir una única instrucción, o todo un bloque de código).

Nota: Rust también nos hará saber si en una instrucción `match` no estamos contemplando todas las posibilidades. Ya hasta me voy a desmayar.

3.11.6 En común

Como cierre de los 3 controles de flujo antes mencionados, me gustaría comentarles que al ser siempre bloques de código lo que sigue de las instrucciones de un controlador de flujo, entonces pueden sacar el máximo provecho del lenguaje al utilizar los valores de retorno de los bloques, pues aquí también aplica, por ejemplo:

```

1 fn is_leap(year: i32) -> bool {
2     if year%4 == 0 && year%100 != 0 {
3         true
4     } else {
5         year%400 == 0
6     }
7 }
8
9 fn main() {
10     if is_leap(2100) {
11         println!("Es_bisiesto!");
12     } else {
13         println!("No_es_bisiesto:");
14     }
15 }

```

Sólo deben de saber que cuando se utiliza este superpoder con el operador de asignación (como `let x = if...`), lo que esté a la derecha debe terminar con punto y coma (en este caso, el bloque de código del controlador de flujo).

Sólo hay que ser cuidadoso con que todos los bloques de código regresen un valor del mismo tipo.

3.12 Vectores

Los vectores son una estructura útil en `rust` porque son como un arreglo pero dinámico. Esto permite constuirlos al tiempo de la ejecución en lugar de hacerlo al momento de la compilación.

La declaración de un vector se hace con la estructura `Vec`, y su método estático `new`

```

1 fn main() {
2     let v: Vec<i64> = Vec::new();
3 }

```

Nota: Carlos, ¿Qué diablos estás haciendo? ¿Qué es ese tipo? Que no cunda el pánico. Si no lo ponen, no va a compilar. Pronto sabrán por qué.

Sin embargo, usarán a menudo un *macro* que permite la inicialización de vectores con varios valores

```
1 fn main() {  
2     // Un vector inicializado  
3     let v = vec![1, 2, 3];  
4 }
```

Este macro también permite inicializar

```
1 fn main() {  
2     // Contiene 10 veces 1.23  
3     let v = vec![1.23; 10];  
4     // Sacamos el segundo elemento  
5     println!("{}", v[1]);  
6 }
```

Nota: A los elementos del vector se accede de la misma forma que a los elementos de un arreglo.

Por supuesto está, los vectores no están sujetos a contener exclusivamente números, uno puede poner cualquier tipo de objeto dentro siempre y cuando éstos sean todos del mismo tipo

```
1 fn main() {  
2     // Contiene 10 veces 1.23  
3     let palabras = vec!["hola", "mundo"];  
4     // Sacamos el segundo elemento  
5     for palabra in palabras.iter() {  
6         print!("{}", palabra);  
7     }  
8     println!("{}", " ");  
9     // Imprime hola, mundo !  
10 }
```

Nota: A esto se le conoce como una clase con tipos genéricos, lo veremos más adelante.

3.12.1 Métodos útiles

- **new:** El constructor del vector.
- **push:** Agrega un elemento al final del vector.
- **pop:** Saca el último elemento del vector. En principio, regresa el elemento extraído, pero hablaremos de eso después.
- **clear:** Borra todos los elementos del vector.
- **len:** Entrega la longitud del vector, en número de elementos.
- **iter:** Genera un iterador que puede ser consumido por un ciclo **for**, por ejemplo.

3.13 Cadenas

No es gratuito que no hemos hablado de cadenas de caracteres hasta ahora. Lo que ocurre es que **Rust** tiene dos tipos de cadenas que se utilizan con frecuencia, **str**, y **String**.

3.13.1 `str`

De los dos, me parece que `str` es el tipo complejo de entender. Una cadena del tipo `str` hace referencia a una secuencia de bytes en algún lugar en memoria con una codificación `utf-8` válida, cuya longitud **es fija** a lo largo de la vida de la cadena. Por este mismo requerimiento es que no podremos declarar variables del tipo `str`, pero sí del tipo `&str`.

Nota: Recuerden, las variables van al *stack*, y por ello su tamaño debe conocerse al momento de la compilación. Poner una referencia a un slice en el *stack* es válido, porque el tamaño de la referencia está definido (una dirección de 8 bytes).

Cuando en los códigos fuente de un programa escrito en **Rust** colocamos una cadena de caracteres entre comillas, esta inmediatamente se interpreta como del tipo `&str`, y su valor se colocará en el *stack*. A final de cuentas, un `str` es una dirección de memoria y una longitud, que respaldan una secuencia de bytes válidos en la codificación antes mencionada.

```
1 fn main() {
2     // Del tipo &str, lo pueden verificar con el compilador
3     let s = "Hola_mundo!";
4     println!("{}", s);
5 }
```

3.13.2 `String`

El tipo `String` es una cadena de caracteres almacenada en el *heap*, que también cumple con ser una secuencia de bytes válidos en codificación `utf-8`. Al ser almacenada en el *heap*, entonces esta estructura tiene dueño, la variable en donde se almacena por primera vez. La utilidad viene de que en la mayoría de las ocasiones necesitamos cadenas dinámicas, y por su ubicación de memoria ésta es la cadena a elegir.

Es muy complicado ir construyendo el lenguaje de programación sin tener conceptos con definiciones cíclicas, por así decirlo, así que por segunda vez utilizaremos algo y no preguntaremos qué es lo que hace. El método `String::from` construye cadenas a partir de objetos del tipo `&str`, listo. No más preguntas.

```
1 fn main() {
2     let mut a = String::from("Hola");
3     let b = "_mundo!";
4     // Esta instruccion tiene su magia
5     a = a + b;
6     println!("{}", a);
7 }
```

La concatenación de cadenas opera siempre con una cadena del tipo `String` a la izquierda, y una del tipo `&str` a la derecha. Eso es porque, por cuestiones de eficiencia, **Rust** *mueve* el contenido de la cadena a la izquierda del operador de la suma a un nuevo lugar en el *heap* donde quepa esa cadena, más la cadena que está a la derecha del operador pero ésta sólo es copiada, no movida.

Nota: Si esta concatenación se quisiera hacer con dos elementos del tipo `String` (es decir, con la declaración `let b = String::from(" mundo!");`), bastaría con escribir la concatenación como `a + &b`, pues resulta que **Rust** puede ver un `&String` como un `&str`

3.13.3 Métodos útiles de las cadenas

Entre los métodos más útiles de las cadenas, se encuentra la siguiente lista:

- **new:** El constructor de una cadena vacía.
- **from:** Es un método que de momento diremos que toma cualquier estructura que pueda interpretarse como un `String`, y lo convierte en un string (por ejemplo `&str`).
- **len:** Regresa la longitud de la cadena en **bytes**.

- **contains**: Indica si la cadena contiene el patrón proporcionado, e.j. `nombre.contains("lopez")`
- **chars**: Regresa un iteraor sobre los caracteres de la cadena.
- **split**: Poderosísimo, para separar la cadena en **tokens**, donde el separador es proporcionado como un caracter, o como una cadena, e.j. `nombre.split(' ')`.
- **into_string**: convierte un `&str` en un `String`

3.14 Los macros `print`, `println` y `format`

Por último, hablaremos de la utilidad de los macros `print` y `println` que sirven para imprimir cadenas sin y con una nueva línea (el carácter `\n`) al final, respectivamente. Lo principal que tienen que saber es que, como buen macro que es, éste no compilará si el número de argumentos adecuado no es proporcionado, pues en la expansión del macro se busca que haya mismo número de llaves de apertura y cierre que parámetros.

Para imprimir el valor de *cosas*, se utiliza entonces `{}`.

```
1 fn main() {
2     let (x, y) = (0, 1);
3     println!("Arg_1_{}", Arg_2_{}", x, y);
4 }
```

A continuación les daré una lista de posibilidades a hacer los macros anteriores. Usaré siempre `println`, aunque sepan que aplica para ambos. Todos los formatos de la siguiente lista van anteceditos por dos puntos.

- Formatos **x**, **X**: Sirven para imprimir un hexadecimal con los caracteres en minúsculas y mayúsculas de manera correspondiente. Se puede anteceder por `#` para imprimir también la notación `0x`.

```
1 fn main() {
2     println!("{:X}", 42); // Imprime 0x2A
3 }
```

En el caso de recibir enteros negativos, este formato imprime el equivalente hexadecimal a la representación *2's complement*.

- Formato **b**: Sirve para imprimir un binario. Se puede anteceder por `#` para imprimir también la notación `0b`.

```
1 fn main() {
2     println!("{:b}", 5); // Imprime 101
3 }
```

En el caso de recibir enteros negativos, este formato imprime el equivalente binario a la representación *2's complement*.

- Formatos **e**, **E**: Sirven para imprimir un flotante en notación científica, donde 10^k es representado como `ek` y `Ek` respectivamente.

```
1 fn main() {
2     println!("{:e}", 100); // Imprime 1e2
3 }
```

- Formato **p**: Imprime la dirección de memoria de una variable. No es una regla dura, pero verán usualmente la impresión en hexadecimal de dicha dirección. Esto sólo funciona con referencias.

```
1 fn main() {
2     let x = 10;
3     println!("{:p}", &x); // No se que imprime :v pero es una direccion
4 }
```

Adicional a esto, existen las siguientes reglas usuales de formateo de texto

- Variables con nombres: El formato de cadenas admite nombres específicos de variables dentro de las llaves, por si se busca hacer lo siguiente

```
1 fn main() {
2     println!("{variable}", variable = 3);
3 }
```

- Variables con índice: El formato de cadenas admite una numeración también para imprimir en repetidas ocasiones los argumentos proporcionados.

```
1 fn main() {
2     // Esto imprime mundo, hola, mundo
3     println!("{1},_{0},_{2}", "hola", "mundo");
4 }
```

- Ancho: un poco complicado de explicar, pero el ancho de la impresión de las cadenas también puede ser controlado. La notación es aproximadamente la siguiente

$\underbrace{*}_{\text{Símbolo}}$ $\underbrace{<}_{\text{Operador de alineación}}$ $\underbrace{7}_{\text{Ancho}}$

En donde el símbolo puede ser algún `char`, que se encargará de rellenar la cadena con los caracteres necesarios para alcanzar el ancho, los operadores son `<` para alinear a la izquierda, `>` para alinear al centro y `^` para alinear al centro, y luego el ancho total de la cadena.

```
1 fn main() {
2     // Imprimiremos -xxxxxxhola-
3     println!("-{:x>10}-", "hola");
4 }
```

Por defecto, el espacio se utiliza como carácter de relleno, y la alineación va a la derecha para números, izquierda para cadenas.

- Precisión: En el caso de flotantes, se puede especificar la precisión con la que debe imprimirse si se anota un número después de un punto.

```
1 fn main() {
2     // Imprimiremos 10.2000000
3     println!("{:.7}", 10.2);
4 }
```

Esto se puede combinar con los operadores anteriores, siempre y cuando la notación de precisión vaya al final.

Como una nota de conclusión, el macro `format` hace lo mismo que `print`, pero en lugar de acudir a la salida estándar, regresa un `String` con el contenido del formato.

```
1 fn main() {
2     let s = format!("Valgo_{0}", 10);
3     println!("{}", s);
4 }
```

4 Uso avanzado del lenguaje

4.1 cargo

Hasta ahora hemos utilizado `rustc` para compilar nuestros códigos fuente en `Rust`, pero el lenguaje es mucho, mucho más que eso. Así como `python` cuenta con `pip`, y `nodejs` cuenta con `npm`, `rust` cuenta con su propio gestor de paquetes, conocido como `cargo`. Veremos unas cuantas de las *monadas* que `cargo` puede hacer.

4.1.1 crates y Cargo.toml

Una de las principales funciones de **cargo** es generar algo que se le conoce como un **crate**, que es como en **rust** se le denominan a las bibliotecas.

Nota: Esto va con una pequeña estrellita, porque en **rust** hay bibliotecas y aplicaciones, por así decirlo.

Para crear un **crate**, ejecutamos el siguiente comando

```
cargo new nombre_de_la_app
```

Si en lugar de una app queremos una biblioteca, debemos añadir la bandera **--lib** (por defecto, es como si la bandera **--bin** estuviera en la ejecución del comando). Esto va a generar un fólder que contiene 2 archivos

- **Cargo.toml**: Un archivo del tipo **toml** (*Tom's Obvious Minimal Language*) que sirve para especificar configuraciones y dependencias del *crate*.
- **src**: Un fólder que contiene como archivo el código fuente **main.rs**, que es un hola mundo de **Rust**.

Nota: En realidad son 4 archivos, pero los otros 2 son de **git**, conocidos como **gitignore** y el fólder de cambios de **git**. Son para llevar un control de versionamiento del código.

Es de notarse también que esta estructura aplica para la aplicación, no para la biblioteca, pero de ello no nos preocuparemos en este curso.

La estructura interna del **Cargo.toml** se debe ver algo así

```
1 [package]
2 name = "nombre_de_la_app"
3 version = "0.1.0"
4 authors = ["Carlos_Malanche_<malanche@hfhciencias.unam.mx>"]
5 edition = "2018"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
```

En este archivo, como pueden ver, viene el nombre de la aplicación, su versión, los autores, la edición (no le muevan) y las dependencias. Además, viene una amigable liga para que vean que otro tipo de cosas se pueden poner aquí. Por esa misma razón es que no veremos la lista exhaustiva en este momento.

Ahora bien, si está en su interés compilar el código fuente del **crate** y ejecutarlo, **cargo** tiene otros dos subcomandos, **build** y **run**. Para compilar, basta con que se paren en el directorio donde se encuentra **Cargo.toml** y ejecuten **cargo build**. Les saldrá algo del siguiente estilo

```
carlos@supercompu:~/nombre_de_la_app> cargo build
  Compiling nombre_de_la_app v0.1.0 (/home/carlos/nombre_de_la_app)
  Finished dev [unoptimized + debuginfo] target(s) in 1.40s
```

En efecto, acaban de construir la aplicación sin optimizaciones y con información de *debug*, es decir, símbolos adicionales que se cargan en el binario para dar trazabilidad a los errores que pudieran surgir (no de compilación, errores humanos). Si quisieran construir la aplicación optimizada y final, que aprovecha todos los recursos del sistema, tienen que colocar la bandera **--release** durante la compilación.

Ahora, ¿Quieren correr su aplicación? Hay de dos. Haber ejecutado **cargo build** generó un nuevo fólder, de nombre **target**, que dentro contendrá 2 carpetas normalmente (**debug** y **release**), dentro de las cuales encontrarán los respectivos binarios producidos por la compilación, que llevarán el nombre de la aplicación (en este ejemplo, **nombre_de_la_app**). Así que pueden ejecutarlo como **./target/debug/nombre_de_la_app**, o si prefiere, ejecutando el comando **cargo run**.

Nota: Ejecutar `cargo run` es más recomendable, porque en caso de no haber construido la aplicación, o en el caso de haber hecho algún cambio al código fuente, `cargo` detecta los cambios en automático y vuelve a compilar nuestra preciosa aplicación.

4.1.2 Modularidad

Ahora, ¿Por qué querríamos hacer esto? Hablaremos de una buena práctica de la ingeniería del software, conocida como *modularidad*. La meta de cualquier ingeniero del software que se respete es generar códigos que no sólo funcionen, pero que además sean fáciles de mantener, entender y mejorar. Además de las buenas prácticas de nombramiento de variables, indentación y demás, la *modularidad* consiste en distribuir los distintos componentes del código en una estructura del árbol de archivos que sea intuitiva, teniendo un efecto directo en el número de líneas máximo que se encuentra en un archivo al mismo tiempo. Regresemos a nuestra estructura `Punto` que usamos como ejemplo cuando descubrimos las estructuras. Si esta estructura va a ser utilizada por todo un proyecto, valdría la pena que se encuentre en su propio archivo.

Así, llegaremos a la siguiente estructura de código

```
src
├── main.rs
├── punto.rs
└── Cargo.toml
```

Ahora, en `punto.rs`, colocaremos lo siguiente

```
1 // Ahora si ponemos el "Pub", para indicar que es visible fuera del modulo
2 pub struct Punto {
3     pub x: f64,
4     pub y: f64,
5     pub z: f64
6 }
```

Nota: La visibilidad de cada elemento de la estructura será necesaria si queremos leer o alterar los campos directamente. Es una práctica común permitir sólo la gestión de las variables miembro por medio de los métodos, pues una estructura siempre tiene acceso a sus propios miembros

Y por último, podemos utilizar nuestro módulo `punto` si lo declaramos desde `main.rs`

```
1 // Incluimos el modulo punto, que es el archivo punto.rs
2 mod punto;
3 // Para ahorrarnos el escribir punto::Punto por todos lados
4 use punto::Punto;
5
6 fn main() {
7     let p = Punto{x: 0.0, y: 0.0, z: 0.0};
8     println!("{}", p.x);
9 }
```

Nota: Algo importante a contemplar, es que la visibilidad va por módulos: entre elementos de un mismo módulo, la visibilidad es absoluta. Por eso es que cuando utilizamos un único archivo de `rust`, no necesitamos la palabra `pub`.

4.1.3 Documentación

`Rust`, como el lenguaje de programación superrobusto que es, tiene integrado un mecanismo de documentación automática. Para ello, existe el comando `cargo doc`, que genera dentro de `target` una estructura de archivos `html`, `css` y `js` que son el fundamento de una muy estructurada documentación, que es estándar a través de todo el lenguaje.

Es el momento pertinente para comentarles que **Rust** mantiene la documentación de todas sus bibliotecas externas en el dominio docs.rs. Cualquier biblioteca que puedan descargar con **cargo**, tendrá su documentación en este sitio. Si quieren acceder a la documentación del crate **crate_ejemplo**, basta con que pongan en su navegador la dirección https://docs.rs/crate_ejemplo, y serán redireccionados al inicio de la documentación.

Claro está, qué clase de curso sería este si no les enseño lo básico para generar ustedes su propia documentación

4.1.4 Documentando funciones, estructuras, variables y enums

Para documentar un elemento *desde fuera*, existe la notación `///`. Esto va a generar un comentario que puede ser interpretado como *markup language*. Veamos el siguiente ejemplo

```
1 /// La mejor funcion del **universo**
2 fn main() {
3     println!("Hola_mundo!");
4 }
```

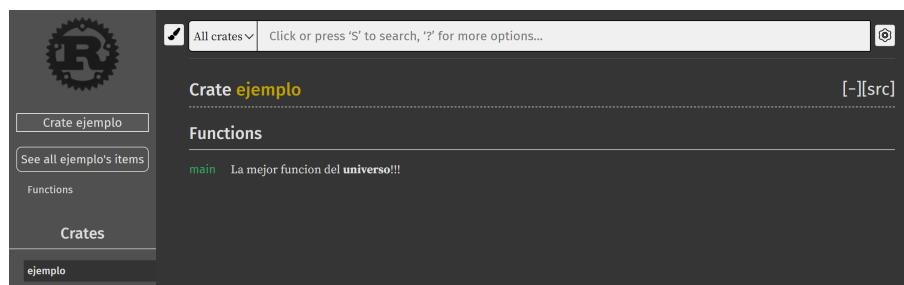
Éste es el archivo `src/main.rs` de un *crate* de nombre **ejemplo**. Al ejecutar **cargo doc**, se generará dentro de **target** una carpeta que lleva por nombre **doc**. Dentro, encontrarán una carpeta que lleva el nombre del crate, **ejemplo** en nuestro caso, y donde finalmente se encuentra el archivo `index.html`. Si uno da doble click a ese archivo, se abrirá el navegador por defecto, desplegando algo como lo siguiente:



Ahora, existe otro tipo de comentario que sirve para documentar desde dentro

```
1 fn main() {
2     /// La mejor funcion del **universo**!!!
3     println!("Hello_world!");
4 }
```

Esto, derivará en la siguiente documentación



Este tipo de documentación es de utilidad dentro de los módulos antes descritos.

4.2 Parámetros genéricos

Tenemos estructuras que tienen un comportamiento extraño, ¿No es así? Como es posible que exista lo siguiente

```
1 fn main() {
2     let palabras = vec!["hola", "mundo"];
3     let numeros = vec![1, 2];
4 }
```

Estamos ante una situación extraña, porque hemos dicho que cada variable en `Rust` tiene un tipo fijo, sin embargo ahí tenemos dos vectores que difieren en contenido (uno tiene elementos del tipo `&str`, el otro de tipo `integer`). ¿Cómo es esto posible? Pues les comento que existe un concepto en `Rust` conocido como parámetros genéricos, que es cuando hacemos una especie de *plantilla* para definir varias clases con diferentes tipos. Si utilizamos nuestro famoso truco del compilador para obligar a que se nos comunique el tipo real que regresa la instrucción a la derecha de las asignaciones, encontraremos que `palabras` tiene tipo `std::vec::Vec<&str>` y `numeros` tiene tipo `std::vec::Vec<{integer}>`. Una estructura templada se declara de la siguiente forma

```
1 struct Templado<S> {
2     nombre: String,
3     contenido: S
4 }
5
6 fn main() {
7     // Ahora, declaramos una instancia con un tipo concreto
8     let _t = Templado {
9         nombre: String::from("Carlos"),
10        contenido: 1.0f64
11    };
12 }
```

Recordemos que el compilador es listo, así que `rust` determinará de manera automática el tipo concreto de la estructura templada `Templado<S>`, donde `S` será `f64` en el caso de la variable `_t`. Cabe destacar que se puede declarar más de un tipo concreto de la estructura templada, sin esto causar problemas

```
1 struct Templado<S> {
2     nombre: String,
3     contenido: S
4 }
5
6 fn main() {
7     let _tf = Templado {
8         nombre: String::from("Carlos"),
9         contenido: 1.0f64
10    };
11    let _ts = Templado {
12        nombre: String::from("Carlos"),
13        contenido: "funciona!"
14    };
15 }
```

En donde `_tf` tendrá el tipo `Templado<f64>` y `_ts` tendrá el tipo `Templado<&str>`. El compilador generará las estructuras concretas que encuentre necesarias durante la compilación, de manera automática.

Ocurre una situación curiosa, sin embargo. ¿Cómo es que haremos un bloque `impl` para ésta estructura? Tenemos dos opciones. Podemos hacer una declaración para un caso concreto de la estructura templada

```
1 impl Templado<String>{
2     fn haz_algo(&self) {
3         println!("No sirvo de nada!");
4     }
5 }
```

O, podemos utilizar un parámetro genérico para la palabra `impl`

```
1 impl<T> Templado<T>{
2     fn haz_algo(&self) {
3         println!("No sirvo de nada!");
4     }
5 }
```

Si nos vamos por la primer opción, el método `haz_algo` estará sólo definido para el caso en que nuestra estructura templada

tenga como parámetro genérico un `String`. En el segundo caso, el bloque `impl` es a su vez genérico, lo que quiere decir que `rust` construirá las funciones que hay adentro para cada tipo del que se declare una estructura.

Nota: Existen las funciones templadas, o con parámetros genéricos también. Ya saldrán a lo largo del curso, porque de momento no contamos con las herramientas necesarias para entenderlas.

4.3 Enums *recargados*

Volvemos a los *enums*, pues estos van a probar ser una de las cosas más poderosas que `Rust` tiene. Es todo un emblema del lenguaje. Resulta que cada variante de un `enum` puede ser una tupla, e incluso una estructura *anónima* (léase, un conjunto de variables de distintos tipos, cada una con su nombre, pero juntas no describen una estructura independiente).

```
1 // El usize dentro de EstadoTienda::Abierta representa un aforo
2 enum EstadoTienda {
3     Abierta(usize),
4     Cerrada
5 }
6 fn main() {
7     let et = EstadoTienda::Abierta(19);
8     match et {
9         EstadoTienda::Abierta(aforo) => {
10             println!("Tienda_abierta_con_aforo_de_{}_persona(s)", aforo);
11         },
12         EstadoTienda::Cerrada => println!("La_tienda_esta_cerrada_(:")
13     }
14 }
```

Es cuestión de que se acostumbren a la notación, pero en la instrucción `match` se puede dar un alias a cada elemento de la tupla. Puede hacerse, como les comenté, una estructura anónima también

```
1 enum Alumno {
2     Oyente{
3         nombre: String,
4         numero_de_cuenta: u32
5     },
6     Inscrito {
7         nombre: String,
8         numero_de_cuenta: u32,
9         numero_de_lista: u8
10    }
11 }
12
13 fn main() {
14     let alumno = Alumno::Inscrito {
15         nombre: String::from("Carlos"),
16         numero_de_cuenta: 123_456_789,
17         numero_de_lista: 0
18     };
19     match alumno {
20         Alumno::Inscrito{nombre, numero_de_cuenta: _, numero_de_lista} => {
21             println!("{}",inscrito_en_la_lista:v_({})", nombre, numero_de_lista);
22         },
23         Alumno::Oyente{nombre, numero_de_cuenta: _} => {
24             println!("{}",no_obtendra_calificacion_(:", nombre);
25         }
26     }
27 }
```

Nota: Para evitarme un par de warnings (aunque no todos), en el match he *renombrado* algunas de las variables de la estructura anónima a se un guión bajo. Esto le indica a **rust**, como ya saben, que no usaré las variables en el bloque de código que continúa.

Cabe destacar que tanto las estructuras anónimas, como las tuplas y las variantes constantes pueden coexistir en un **enum**, sin mayor consecuencia.

Nota: En el siguiente ejemplo, verán en uso el macro **panic**. Lo que hace ese macro, es detener la ejecución del programa a toda costa, efectivamente, entrando en un pánico y descontrol absoluto. Úsese con cuidado.

```
1 enum Opcion {
2     Algun(String),
3     Nada
4 }
5
6 struct Persona {
7     nombres: Vec<String>
8 }
9
10 impl Persona {
11     fn new(nombres: Vec<String>) -> Persona {
12         if nombres.len() == 0 || nombres.len() > 2 {
13             panic!("Oh_noes!");
14         }
15         Persona{nombres}
16     }
17
18     fn primer_nombre(&self) -> String {
19         self.nombres[0].clone()
20     }
21
22     fn segundo_nombre(&self) -> Opcion {
23         if self.nombres.len() == 1 {
24             Opcion::Nada
25         } else {
26             Opcion::Algun(self.nombres[1].clone())
27         }
28     }
29 }
30
31 fn main() {
32     let persona = Persona::new(vec![String::from("Carlos"), String::from("Gerardo")]);
33     match persona.segundo_nombre() {
34         Opcion::Algun(nombre) => println!("El_segundo_nombre_es_{}", nombre),
35         Opcion::Nada => println!("No_hay_segundo_nombre!")
36     }
37 }
```

Oh por dios, todo un mundo de posibilidades. ¿Y saben qué es lo mejor? Los **enums** también pueden ser templados. Así que podemos generalizar nuestro **enum Opcion**

```
1 enum Opcion<T> {
2     Algun(T),
3     Nada
4 }
```

Pues, ¿Qué creen? Es de semejante utilidad esta estructura, que ya viene cargada en **rust** como **enum** estándar, bajo el nombre **Option**.

4.3.1 El enum Option

Éste luce masomenos así

```
1 enum Option<T> {  
2     Some(T),  
3     None  
4 }
```

Habrán muchos, pero verdaderamente muchos métodos y funciones que regresarán este `enum`. El `enum` viene acompañado de muchos métodos que son de utilidad, por ejemplo

- `unwrap`: Regresa el valor contenido en `Some(T)`. En caso de ser la variante `None`, el programa entra en pánico.
- `is_none`: Regresa un booleano que nos indica si dentro hay una variante `None`.
- `is_some`: Regresa un booleano que nos indica si dentro hay una variante `Some(T)`.

4.3.2 El enum Result

Bajo la misma mentalidad, el manejo de errores se encuentra definido bajo un `enum bi-templado` de nombre `Result`, cuya definición se ve masomenos así

```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E)  
4 }
```

Un ejemplo de uso de dicho `enum`, sería el siguiente

```
1 fn main() {  
2     match String::from_utf8(vec![0xc3, 0xb1]) {  
3         Ok(cadenita) => println!("La_cadena_es_{}", cadenita),  
4         Err(_error) => {  
5             println!("Los_bytes_no_son_utf-8:");  
6         }  
7     }  
8 }
```

Nota: Ese par de bytes representan la letra ñ en `utf-8`

`Result` se encuentra dotado de muchos métodos también. Una lista no exhaustiva es la siguiente

- `unwrap`: Regresa el valor contenido en `Ok(T)`. En caso de ser la variante `Err(E)`, el programa entra en pánico.
- `unwrap_err`: Regresa el valor contenido en `Err(E)`. En caso de ser la variante `Ok(T)`, el programa entra en pánico.
- `is_ok`: Regresa un booleano que nos indica si dentro hay una variante `Ok(T)`.
- `is_err`: Regresa un booleano que nos indica si dentro hay una variante `Err(E)`.

Saldrán muchas dudas al respecto del uso de estos `enums`, así que recuerden que están todos los manuales en línea con la documentación de todos los métodos asociados a estos `enums`.

4.4 Características

El último concepto complejo del curso. ¿Qué es una característica?

Una característica equivale, en los otros lenguajes de programación, a la herencia de clases. Una característica describe los métodos que debe poseer una estructura, o un `enum`. Es relativamente simple, y para ello se utiliza la palabra `trait`. Vamos a hacer un ejemplo sencillo. Absolutamente todos sabemos que no puede haber mascota sin nombre. Por lo tanto, podemos definir una *característica* de nombre `Mascota`, que obligue a cualquiera que la `implemente` a poder responder con un nombre. Para eso existirá la instrucción `impl Mascota for Tipo`, donde reemplazaremos la palabra `Tipo` por la estructura que se desee sea una mascota.


```

1 trait Mascota {
2     /// Toda mascota tiene un nombre, obviamente
3     fn nombre(&self) -> String;
4 }
5
6 struct Perro {
7     nombreeee: String
8 }
9
10 struct Gato {
11     nom: String
12 }
13
14 impl Mascota for Perro {
15     fn nombre(&self) -> String {
16         self.nombreeee.clone()
17     }
18 }
19
20 impl Mascota for Gato {
21     fn nombre(&self) -> String {
22         self.nom.clone()
23     }
24 }
25
26 fn main() {
27     let (p, g) = (Perro{nombreeee: String::from("Lomito")}, Gato{nom: String::from("Michi")});
28     println!("El_perro_se_llama{}", p.nombre());
29     println!("El_gato_se_llama{}", g.nombre());
30 }

```

Una estructura puede implementar múltiples características. Me gustaría resaltar por qué existe este concepto en **rust**:

Resulta que se busca que el lenguaje sea fuertemente tipado, pero no obstante permita a las estructuras de parámetros genéricos existir, sin conocimiento del tipo final que se implementará (tal es el caso de las bibliotecas que definen estructuras con parámetros genéricos). Resulta entonces conveniente decirle a **rust** que, pese a no conocer el tipo concreto de una implementación, puede saber qué comportamientos mostrará. Para ello se crearon los **traits**. Existen varios por defecto, mencionaremos algunos de ellos.

4.4.1 Características comunes

- `std::fmt::Display`: Permite a un objeto ser impreso, es decir, convertirse en cadena de caracteres.
- `std::clone::Clone`: Permite que la estructura o **enum** sea clonado, es decir, se genere una copia idéntica e independiente de la instancia que llama, a través del método `clone`.
- `std::io::Write`: Indica que se puede escribir una serie de bytes a esta estructura.
- `std::io::Read`: Indica que se puede leer una serie de bytes de esta estructura.

4.4.2 Nota importante

Resulta importante destacar que los **traits**, a pesar de estar implementados para alguna estructura, no serán detectados por el compilador si no se encuentran en el módulo en el que búscan usarse. Explico.

Si yo defino un trait que se llama **Mascota**, y la definición del trait está en un lado distinto al **Perro**, quien importe **Perro** también tendrá que importar **Mascota** para que se puedan utilizar los métodos que se han implementado por la característica **Mascota**. La razón de fondo por la que esto ocurre excede un poco los fines del curso, así que sólo recuérdalo como regla.

Nota: Se les hace saber que esto no aplica para algunos de los traits que vienen por defecto con `rust`, para facilitar la lectura del código.

4.5 Escritura de archivos, entrada y salida estándar

Carlos, pero no nos has enseñado cómo escribir archivitos y todas esas cosas útiles para nuestros futuros cálculos que haremos, como científicos numéricos absolutos. Es cierto, pero vengo a subsanar esa carencia. La salida estándar ya la tenemos bien dominada con el uso de los macros `print` y `println`. ¿Pero la entrada?

4.5.1 Entrada estándar

La entrada estándar es, pues, aquél stream de *bytes* que le llega a un comando de *unix* al usar un *pipe*. Es decir, estamos hablando de esa famosa interacción entre comandos. Es la secuencia que puede recibir un programa mientras éste se ejecuta, y nos permite hacer esta comunicación, normalmente, por medio del shell.

En Rust, para esta finalidad, existe la función `std::io::stdin`, que regresa una estructura del tipo `Stdin` que permite comunicarnos con la salida estándar. En el uso avanzado del lenguaje veremos lo que es una *característica* en Rust. De momento sólo sepan que es un conjunto de funciones que son dotadas a una estructura, en este caso la característica se llama `Stdin`, que nos dará el método más importante: `read_line`.

```
1 fn main() {
2     println!("Ingrese un nombre por favor");
3     let stdin = std::io::stdin();
4     let mut nombre = String::new();
5     stdin.read_line(&mut nombre).unwrap();
6     // Imprimimos, pero cortando la nueva línea del final
7     println!("Hola, {}!", nombre.trim());
8 }
```

Añado que el tipo `&str` viene acompañado del método `parse`, que intenta convertir la cadena en otro tipo. Esto regresa un enum `Result`, así que un ejemplo de su uso, para leer un `i32`, es el siguiente:

```
1 fn main() {
2     println!("Ingrese un entero por favor");
3     let stdin = std::io::stdin();
4     let mut value = String::new();
5     stdin.read_line(&mut value).unwrap();
6     // Ahora intentamos convertirlo en un i32, para lo cual
7     // le ayudamos al compilador a adivinar el tipo
8     let num: i32 = value.trim().parse().unwrap();
9     // Imprimimos, pero cortando la nueva línea del final
10    println!("Hola, {}!", num);
11 }
```

Nota: `parse` es una función templada, por si se lo andaban preguntando.

4.5.2 Escritura de archivos

Ya estamos llegando al momento culminante de Rust, donde usaremos mucho de lo que se ha visto. Se presenta ante nosotros la estructura `std::fs::File`, que está para representar un archivo del árbol de archivos del sistema operativo. Ésta estructura implementa las características `Read` y `Write`. En el caso de la escritura de archivos, nos auxiliaremos del método `write_all`, que permite escribir una secuencia de bytes. Utilizaremos también el método `create` para intentar crear un archivo en el filesystem.

```

1 use std::fs::File;
2 use std::io::Write;
3
4 fn main() {
5     let content = "Hola_mundo!";
6     match File::create("archivo.txt") {
7         Ok(mut f) => {
8             f.write_all(content.as_bytes()).unwrap();
9         },
10        Err(e) => {
11            panic!("{}", e);
12        }
13    }
14 }

```

Tras la ejecución de este bloque, se habrá generado el archivo `archivo.txt`, que contiene la cadena `Hola mundo!`. Sencillo, ¿No? Lo único que deben tener siempre presente, es que el archivo sólo puede recibir un stream de bytes.

Nota: Nótese la importación del *trait* `Write`, para que pueda usar dichos métodos de la estructura `File`.

4.5.3 Lectura de archivos

Ahora, para la lectura, tenemos un par de opciones. La primera, es añadir todo el contenido de nuestro archivo a un vector de bytes, o en nuestro caso, a una cadena.

```

1 use std::fs::File;
2 use std::io::Read;
3
4 fn main() {
5     let content = match File::open("archivo.txt") {
6         Ok(mut f) => {
7             let mut content = String::new();
8             f.read_to_string(&mut content).unwrap();
9             content
10        },
11        Err(e) => {
12            panic!("{}", e);
13        }
14    };
15    println!("{}", content);
16 }

```

Nota: Similar a la nota anterior, noten la importación de `Read` para poder acceder al método `read_to_string` que tiene `File` por implementar el mencionado *trait*.

La segunda opción, es leer el archivo línea por línea, lo cuál muchas veces resulta muy conveniente. Sin embargo, al *leer línea por línea* ser un concepto que sólo existe para archivos codificados como `utf-8`, éste método no está en el *trait* `Read`. Para ello, existe la estructura `BufReader`, con el *trait* `BufRead`, que permite leer un archivo completo por medio de un iterador.

```

1 use std::fs::File;
2 use std::io::{BufReader, BufRead};
3
4 fn main() {
5     match File::open("archivo.txt") {
6         Ok(f) => {
7             let reader = BufReader::new(f);
8             for line in reader.lines() {
9                 // Cada linea puede contener un error
10                let line = line.unwrap();
11                println!("{}", line);
12            }
13        },
14        Err(e) => {
15            panic!("{}", e);
16        }
17    };
18 }

```

Ahora sí, pueden auxiliarse de todos los métodos que tiene la estructura `str` para que puedan extraer de cada línea múltiples campos, números, etc.

4.6 Cerraduras

Ahora sí nos vamos despidiendo. Las cerraduras las menciono por la importancia que tendrán en el uso de otros *crates*. Una cerradura no es más que una función en una variable. Es algo conocido como una `lambda` en otros lenguajes de programación. La sintaxis está ejemplificada en el siguiente código

```

1 fn main() {
2     // Cerradura con todas las anotaciones
3     let suma = |a: i32, b| -> i32 {
4         a + b
5     };
6     // Cerradura sin anotaciones ni argumentos, de una unica instruccion
7     let saluda = || println!("Hola!");
8
9     saluda();
10    println!("{}", suma(1, 2));
11 }

```

Las cerraduras pueden adivinar el tipo de los parámetros. En el caso de la cerradura `suma`, `rust` automáticamente sabe que la variable `b` forzosamente debe poder ser llamada ante el operador `+`, donde se involucra `a` que tiene tipo `i32`. `Rust` mantiene siempre las garantías de un fuerte tipado.

Ahora, podrán preguntarse ¿Por qué esto es de utilidad? Les había comentado que `Rust` es un lenguaje con características de programación funcional. Las cerraduras son un elemento clave en dicho paradigma, y se pueden utilizar para modificar de manera muy ágil estructuras de datos como vectores, sin necesidad de declarar funciones que de manera *global*. En el siguiente ejemplo, a un vector de palabras

```

1 fn main() {
2     let gritador = |s: &&str| format!("{}", !s, *s);
3
4     let values: Vec<String> = vec!["Hola", "Mundo"].iter().map(gritador).collect();
5
6     for value in values {
7         println!("{}", value);
8     }
9 }

```

Analicen con lujo de detalle el código anterior. Es glorioso. La salida serán las palabras del vector original, finalizadas por un signo de admiración. En la sección de *extras* nos encontraremos con varios casos de uso para las cerraduras.

5 Extras

Y bueno, muchachos. No llevamos ni un 50% de todo lo que este lenguaje puede hacer. Pero ya queda en ustedes, si es que les nace el interés, aprender mucho más del lenguaje para usar su absoluto potencial. De momento, les dejaré aquí algunos extra que les serán de ayuda durante el curso.

5.1 Graficando con Rust

Como último detalle que puede serles de utilidad, les muestro una biblioteca de código que sirve para graficar de manera muy elegante y vectorial, `plotters`.

Lo primero que se necesitará para utilizar esta biblioteca, es añadir `plotters` a las dependencias del `textttCargo.toml` del proyecto en donde queremos utilizarlo.

```
1 [dependencies]
2 plotters = "^0.3.0"
```

Si quieren conocer la versión más actual de un `crate`, pueden utilizar el comando `cargo search nombre_del_crate`. Eso consulta el repositorio de *crates* oficial de Rust. Vamos a ir directo a un código, porque como verán, es bastante complejo. Con esto, se puede graficar la función $y = x^2$ en el intervalo $(-1, 1)$. Hay un uso pesado de programación funcional en este código, la explicación de cada línea vendrá adelante.

```
1 use plotters::prelude::*;
2
3 fn main() {
4     let root = SVGBackend::new("ejemplo.svg", (1280, 720)).into_drawing_area();
5     root.fill(&WHITE).unwrap();
6     let mut chart = ChartBuilder::on(&root)
7         .caption("y=x^2", ("sans-serif", 50).into_font())
8         .margin(5)
9         .x_label_area_size(30)
10        .y_label_area_size(30)
11        .build_cartesian_2d(-1f32..1f32, -0.1f32..1f32).unwrap();
12
13    chart.configure_mesh().draw().unwrap();
14
15    chart
16        .draw_series(LineSeries::new(
17            (-50..=50).map(|x| x as f32 / 50.0).map(|x| (x, x * x)),
18            &RED,
19        ))
20        .unwrap()
21        .label("y=x^2")
22        .legend(|(x, y)| PathElement::new(vec![(x, y), (x + 20, y)], &RED));
23
24    chart
25        .configure_series_labels()
26        .background_style(&WHITE.mix(0.8))
27        .border_style(&BLACK)
28        .draw().unwrap();
29 }
```

Uff. Cabe destacar que `plotters` es una biblioteca de dibujo, no de graficado únicamente. Eso la hace una biblioteca exageradamente poderosa. Ahora sí, línea por línea.

```
1 use plotters::prelude::*;
```

En esta instrucción, se utiliza el *wildcard* `*` para importar todo lo que se encuentra en el módulo `prelude` de `plotters`.

```
1 let root = SVGBackend::new("ejemplo.svg", (1280, 720)).into_drawing_area();
2 root.fill(&WHITE).unwrap();
```

Aquí, se guarda en la variable `root` una estructura que será un *Backend*, léase, algo que aceptará los dibujos que haremos con las futuras instrucciones. Podría ser un png, pero preferimos utilizar un svg pues es vectorial. La instrucción siguiente configura un fondo en color blanco.

```
1 let mut chart = ChartBuilder::on(&root)
2     .caption("y=x^2", ("sans-serif", 50).into_font())
3     .margin(5)
4     .x_label_area_size(30)
5     .y_label_area_size(30)
6     .build_cartesian_2d(-1f32..1f32, -0.1f32..1f32).unwrap();
7
8 chart.configure_mesh().draw().unwrap();
```

En este par de instrucciones, se genera una *gráfica* que se guarda en la variable `chart`. La leyenda tendrá por caption $y = x^2$ (sin L^AT_EX), en fuente *sans-serif* de 50 puntos. Además, le decimos que hay 30 puntos de espacio para colocar las etiquetas en ambos ejes. Al final, utilizamos todo esto para construir un plano cartesiano que en x va de -1 a 1, y en y va de -0.1 a 1 (esto le dice a `rust` que todo lo que se dibuje, deberá tener dos coordenadas).

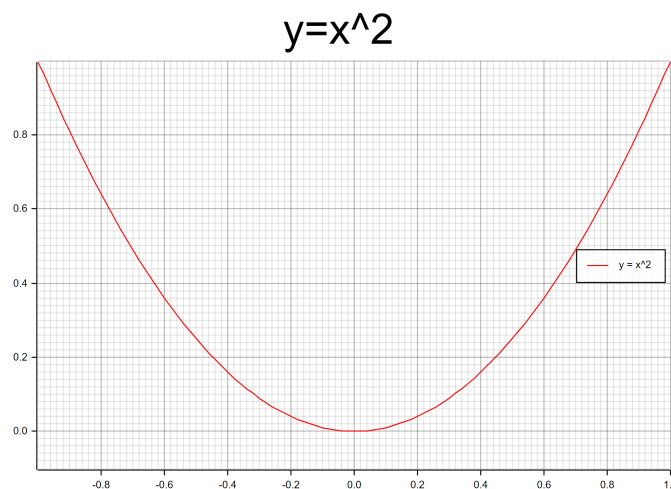
La instrucción de `configure_mesh` se utiliza para hacer una retícula (podríamos añadir más métodos que configuran el estilo de la línea y demás cosas, pero queremos mantener el código simple). Con `draw`, dibujamos la retícula en la gráfica.

```
1 chart
2     .draw_series(LineSeries::new(
3         (-50..50).map(|x| x as f32 / 50.0).map(|x| (x, x * x)),
4         &RED,
5     )).unwrap()
6     .label("y=x^2")
7     .legend(|(x, y)| PathElement::new(vec![(x, y), (x + 20, y)], &RED));
```

Aquí, se le comunica a la gráfica que contendrá una *serie* de puntos que deberán estar unidos por líneas. Lo que sigue, es programación funcional para graficar los puntos (x, x^2) para $x \in [-1, -0.98, -0.96, \dots, 0.98, 1]$. Además añadimos la etiqueta de la serie, que se hace a través de una cerradura.

```
1 chart
2     .configure_series_labels()
3     .background_style(&WHITE.mix(0.8))
4     .border_style(&BLACK)
5     .draw().unwrap();
```

Por último, esta instrucción configura las etiquetas, y hace el dibujo final. El resultado es el siguiente.



Como comentario adicional, no olviden visitar el sitio <https://docs.rs/plotters>